

# claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a08b1f55794109e19.jsonl`  
- SHA-256: `ba3263bb1278e32418ec3bc8f7b25d59288bb324e858d2673dc5f01e05f72720`  
- Source modified: `2026-07-02T02:55:34+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_41c8dc8a-8a4`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

## Transcript

### 1. user (2026-07-02T02:47:11.980Z)

CONTEXT (read carefully):

- You are one auditor in a multi-agent tech-debt/critical-fix audit of the khelsutra-guru multi-repo workspace at C:/Users/avidu/Projects/khelsutra-guru. Today is 2026-07-02. All git repos were just pulled to current origin master/main.
- Repos: badminton-highlight-indexer (the ENGINE, Python, ~1500 tests, org Khelsutra), khelsutra (SPA web/ + marketing site/, TS/React, org avidu), sports-data-collector, rally-annotator, sports-obsreport (small Python repos). Plus two NON-git dirs: "Annotation Setup", khelsutra-screencast.
- HARD RULES: READ-ONLY. Do NOT modify/create/delete any file, do NOT create branches/worktrees, do NOT pip install, do NOT run the full test suite (CI is trusted on master). You MAY run read-only git and gh commands (gh is authenticated), ruff/mypy IF already installed, and fast read-only scripts.
- EVERY claim must cite file:line (or a gh URL). A claim without file:line evidence is a hypothesis - mark it as such or drop it. The codebase drifts between sessions: re-verify anything you believe from docs against actual code.
- OWNER-GATED areas (flag as owner\_gated=true, still report): alpha/serving go-live + Cloudflare dashboard work, Studio P10/P11 (corpus promotion + train/reeval), product/strategy/licensing decisions, real GPU/cloud spend, counsel ToS/DPA, repo rename off "badminton".
- Known-open items from project memory (VERIFY current state, don't trust blindly): engine PR #390 (D1 split main.py, open since 06-26) · audit doc docs/CODE\_CLEANUP\_AUDIT\_2026-06-24.md remaining items D1, D13 (ByteTrack->trackers), Phase-3 (B3/B4/B6/B7/B8/D5/D6/D8-D10/03/04) · docs/DECISION\_SNAPSHOT\_REGRESSION.md next=P1 · docs/GOLDEN\_REGRESSION\_FIXTURES.md backlog M4/P0-rename/01/02 · engine issues #340 (Gemini re-bill), #332 (NSFW preflight), #349 (CPU-bound decode), #384 (CI single-source) · khelsutra issues #97-#114 UX/alpha backlog · DO droplet RETIRED 2026-06-25 (docs may still reference it; api.khelsutra.guru CNAME/CF-Access cleanup was pending).
- Categorize each finding: critical-fix (latent bug/correctness/data-loss/cost-leak), security, tech-debt (structure/duplication/dead code), test-gap, ci-infra, doc-rot, hygiene.
- Severity: P0 = latent bug or cost/data/security risk reachable in normal use; P1 = high-value debt blocking velocity or a stale-open thread; P2 = worthwhile cleanup; P3 = nice-to-have (report max 3 of these).
- Be selective: max ~12 findings, each genuinely worth acting on. Note tracked\_in when an issue/PR/doc already tracks it (tracked is NOT done - still report). Your final message is data for an orchestrator, not prose for a human.

YOUR DIMENSION: code health of the khelsutra repo (SPA + marketing site). Repo:

C:/Users/avidu/Projects/khelsutra-guru/khelsutra.

Check: web/ (React SPA) - TS strictness, test coverage state (vitest), component size (VideoLibrary.tsx just landed in #140), API-client drift vs the engine's current endpoints (compare web/src API calls to C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/main.py + backend/api/ routes - any endpoint the SPA calls that changed shape?); the ENGINE BUNDLE-SYNC convention: the engine vendors a built SPA at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/ui/ +

backend/api/static/app.js - is that bundle STALE vs khelsutra web/ current source (compare build hashes/dates/marker strings)? site/ (marketing) - i18n key parity across the 6 locales (a key-parity check script may exist), dead deploy configs referencing the RETIRED droplet (deploy/, .github/workflows - the parity monitor was removed in #117; check leftovers like cloudflared configs, DEPLOY\_ALPHA.md accuracy, GPU\_VM\_SERVING.md). node\_modules junction gotchas aside, do NOT run npm install.

## 2. user (2026-07-02T02:47:18.860Z)

f9eea14 web: resilient video-summary readers + show fps in the library (#140)  
9223ba9 docs: refresh khelsutra docs for today's storage-track work (P2-P5) (#139)  
fb2f561 web: persistent "FFmpeg not detected" banner (hard-blocker warning) (#138)  
8667dbf P5: SPA storage-medium picker (store a video into your medium) (#137)  
a04a21b docs(tracker): inline the D14 decisions into the P2-P5 rows (#136)  
NEXT\_STEPS.md  
README.md  
deploy  
docs  
locales  
node\_modules  
site  
web

## 3. user (2026-07-02T02:47:19.496Z)

site:  
README.md  
access.db  
make Og.py  
node\_modules  
package-lock.json  
package.json  
public  
schema.sql  
scripts  
server.js  
src  
wrangler.toml  
  
web:  
README.md  
dist  
e2e  
index.html  
node\_modules  
package-lock.json  
package.json  
src  
tsconfig.json  
vite.config.ts  
  
web/src:  
App.tsx  
annotator  
api  
components  
hooks  
i18n  
index.css  
lib  
main.tsx  
test  
vite-env.d.ts

#### 4. user (2026-07-02T02:47:27.600Z)

```
1      {
2        "compilerOptions": {
3          "target": "ES2022",
4          "useDefineForClassFields": true,
5          "lib": ["ES2022", "DOM", "DOM.Iterable"],
6          "module": "ESNext",
7          "skipLibCheck": true,
8
9          "moduleResolution": "bundler",
10         "allowImportingTsExtensions": true,
11         "resolveJsonModule": true,
12         "isolatedModules": true,
13         "moduleDetection": "force",
14         "noEmit": true,
15         "jsx": "react-jsx",
16
17         "strict": true,
18         "noUnusedLocals": true,
19         "noUnusedParameters": true,
20         "noFallthroughCasesInSwitch": true,
21
22         "types": ["vite/client", "node"]
23       },
24       "include": ["src", "vite.config.ts"]
25     }
26
```

#### 5. user (2026-07-02T02:47:28.059Z)

```
1      {
2        "name": "khelsutra-web",
3        "private": true,
4        "version": "0.0.0",
5        "type": "module",
6        "description": "KhelSutra Studio ? operator console SPA (drives the rally indexer engine).",
7        "scripts": {
8          "dev": "vite",
9          "build": "vite build",
10         "preview": "vite preview",
11         "typecheck": "tsc --noEmit",
12         "test": "vitest run",
13         "test:coverage": "vitest run --coverage",
14         "e2e": "playwright test -c e2e/playwright.config.mjs",
15         "screencast": "vite build && playwright test -c e2e/playwright.config.mjs",
16         "screencast:mp4": "node e2e/to-mp4.mjs"
17       },
18       "dependencies": {
19         "@fontsource-variable/noto-sans-devanagari": "^5.2.8",
20         "@fontsource-variable/noto-sans-kannada": "^5.2.9",
21         "i18next": "^26.3.1",
22         "i18next-browser-languagedetector": "^8.2.1",
23         "i18next-icu": "^2.4.3",
24         "react": "^19.0.0",
25         "react-dom": "^19.0.0",
26         "react-i18next": "^17.0.8"
27       },
28       "devDependencies": {
29         "@playwright/test": "^1.56.0",
30         "@tailwindcss/vite": "^4.0.0",
31         "@testing-library/dom": "^10.4.1",
32         "@testing-library/react": "^16.3.2",
33         "@types/node": "^26.0.0",

```

```
34     "@types/react": "^19.0.0",
35     "@types/react-dom": "^19.0.0",
36     "@vitejs/plugin-react": "^4.3.4",
37     "@vitest/coverage-v8": "^3.2.6",
38     "jsdom": "^29.1.1",
39     "tailwindcss": "^4.0.0",
40     "typescript": "^5.7.0",
41     "vite": "^6.0.0",
42     "vitest": "^3.0.0"
43   }
44 }
45
```

## 6. user (2026-07-02T02:47:29.848Z)

```
src/annotator:
VENDOR.md
annotator.smoke.test.ts
annotator.ts
csv.ts
```

```
src/api:
client.test.ts
client.ts
locale.test.ts
types.ts
upload.test.ts
```

```
src/components:
AnnotatePanel.test.tsx
AnnotatePanel.tsx
CompileResult.tsx
FfmpegBanner.test.tsx
FfmpegBanner.tsx
IngestPanel.test.tsx
IngestPanel.tsx
LanguageSwitcher.tsx
QueryBar.examples.test.ts
QueryBar.test.tsx
QueryBar.tsx
RallyGrid.tsx
SelectionFooter.test.tsx
SelectionFooter.tsx
SetupWizard.test.tsx
SetupWizard.tsx
StatusChip.tsx
TapFilters.test.tsx
TapFilters.tsx
VideoLibrary.test.tsx
VideoLibrary.tsx
a4.touch.test.tsx
a8.status.test.tsx
build-error.test.tsx
upload.test.tsx
```

```
src/hooks:
useIngestJob.test.ts
useIngestJob.ts
useRallySelection.ts
useUpload.test.ts
useUpload.ts
```

```
src/lib:
errorText.test.ts
errorText.ts
```

errors.test.ts  
errors.ts  
floor.test.ts  
floor.ts  
format.test.ts  
format.ts  
log.ts  
query.test.ts  
query.ts  
selection.test.ts  
selection.ts  
video.test.ts  
video.ts

src/test:  
setup.ts  
src/annotator/annotator.smoke.test.ts  
src/api/client.test.ts  
src/api/locale.test.ts  
src/api/upload.test.ts  
src/components/a4.touch.test.tsx  
src/components/a8.status.test.tsx  
src/components/AnnotatePanel.test.tsx  
src/components/build-error.test.tsx  
src/components/FfmpegBanner.test.tsx  
src/components/IngestPanel.test.tsx  
src/components/QueryBar.examples.test.ts  
src/components/QueryBar.test.tsx  
src/components/SelectionFooter.test.tsx  
src/components/SetupWizard.test.tsx  
src/components/TapFilters.test.tsx  
src/components/upload.test.tsx  
src/components/VideoLibrary.test.tsx  
src/hooks/useIngestJob.test.ts  
src/hooks/useUpload.test.ts  
src/i18n/catalog-invariants.test.ts  
src/i18n/hygiene.test.ts  
src/i18n/i18n.test.tsx  
src/lib/errors.test.ts  
src/lib/errorText.test.ts  
src/lib/floor.test.ts  
src/lib/format.test.ts  
src/lib/query.test.ts  
src/lib/selection.test.ts  
src/lib/video.test.ts

## 7. user (2026-07-02T02:47:39.172Z)

4611 total  
417 src/App.tsx  
376 src/components/IngestPanel.tsx  
341 src/api/client.ts  
312 src/components/IngestPanel.test.tsx  
233 src/api/client.test.ts  
229 src/components/AnnotatePanel.tsx  
186 src/api/types.ts  
179 src/components/SetupWizard.tsx  
173 src/hooks/useIngestJob.test.ts  
147 src/hooks/useIngestJob.ts  
127 src/components/SetupWizard.test.tsx  
124 src/components/build-error.test.tsx  
120 src/components/SelectionFooter.test.tsx  
114 src/components/QueryBar.tsx  
110 src/components/a8.status.test.tsx  
101 src/components/VideoLibrary.tsx

```

96 src/components/CompileResult.tsx
93 src/components/upload.test.tsx
93 src/api/upload.test.ts
88 src/hooks/useUpload.ts
87 src/components/a4.touch.test.tsx
81 src/components/SelectionFooter.tsx
79 src/components/RallyGrid.tsx
75 src/hooks/useUpload.test.ts

```

## 8. user (2026-07-02T02:47:39.413Z)

```

1      // Typed client for the engine FastAPI. Default = SAME-ORIGIN relative `/api/*` (the
single-origin
2      // appliance reverse-proxies UI + engine behind one origin; in dev Vite proxies `/api` ?
:8000).
3      // Every call carries a correlation id (header + logs) so a journey is traceable end to
end.
4      import { log, newCorrelationId } from "../lib/log";
5      import type {
6          RallySegment,
7          QueryResponse,
8          CompileResult,
9          EngineConfig,
10         EngineCapabilities,
11         ProcessResponse,
12         JobStatus,
13         UploadInitResponse,
14         UploadPartResponse,
15         UploadCompleteResponse,
16         VideoSummary,
17         VideosResponse,
18         AnnotationsResponse,
19         AssetResponse,
20     } from "../types";
21
22     /** Resolve the engine API base. Default same-origin `"/api"`. For a SPLIT deploy ? SPA
on
23     * Cloudflare Pages + the engine on a separate `api.*` cloudflared tunnel
(HOSTING_PLAN.md §2) ?
24     * set `VITE_API_BASE` at build time to the engine's API base **including the `/api`
path**
25     * (e.g. `https://api.khelsutra.guru/api`), and lock the engine's
`RALLY_CORS_ALLOW_ORIGINS` to the
26     * Pages origin + put CF Access in front of both hostnames. A trailing slash is trimmed
so
27     * `` `${base}/process` `` never double-slashes. */
28     export function apiBase(): string {
29         return (import.meta.env.VITE_API_BASE ?? "/api").replace(/\/+$/, "");
30     }
31
32     const BASE = apiBase();
33
34     export class ApiError extends Error {
35         readonly cid?: string;
36         readonly status?: number;
37         readonly body?: string;
38         constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
39             super(message);
40             this.name = "ApiError";
41             this.cid = opts.cid;
42             this.status = opts.status;
43             this.body = opts.body;
44             if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
45         }

```

```

46     }
47
48     interface RequestOpts {
49         method?: string;
50         body?: string;
51         headers?: Record<string, string>;
52         /** Correlation id to thread through this call; auto-generated if absent. */
53         cid?: string;
54     }
55
56     async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
57         const cid = opts.cid ?? newCorrelationId();
58         const url = `${BASE}${path}`;
59         const method = opts.method ?? "GET";
60         log("info", "api.request", { cid, method, url });
61
62         let res: Response;
63         try {
64             res = await fetch(url, {
65                 method,
66                 body: opts.body,
67                 headers: { "content-type": "application/json", "x-correlation-id": cid,
68             ...(opts.headers ?? {}) },
69             });
69         } catch (err) {
70             // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
71             log("error", "api.network_error", { cid, method, url, error: String(err) });
72             throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
73         }
74
75         if (!res.ok) {
76             const body = await res.text().catch(() => "");
77             log("error", "api.http_error", { cid, method, url, status: res.status, body:
78 body.slice(0, 500) });
79             throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
80 res.status, body });
81         }
82
83         const data = (await res.json()) as T;
84         log("info", "api.response", { cid, method, url, status: res.status });
85         return data;
86     }
87
88     /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
89     for
90     * processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-
91     fails 400
92     * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
93     ApiError. */
94     export async function processVideo(
95         videoPath: string,
96         cid?: string,
97         locale?: string,
98         compute?: string,
99     ): Promise<ProcessResponse> {
100         // `locale` (the UI language) is recorded for PMF analytics (which markets use the
101         tool). `compute`
102         // is the compute-picker choice ("local" | "cloud") ? where DETECT runs; omitted ? the
103         server
104         // default. Both optional + omitted when absent (JSON.stringify drops undefined).
105         return request<ProcessResponse>("/process", {
106             method: "POST",
107             body: JSON.stringify({ video_path: videoPath, locale, compute }),
108             cid,
109         });
110     }

```

```

103     }
104
105     /** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
106     export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
107         return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
108     });
109
110     /** List indexed videos, newest first ? the "my videos" library. GET /api/videos
111     [main.py:447].
112     * Reload-survival: a past match can be reopened without re-processing. Returns [] on
113     an empty box. */
114     export async function listVideos(cid?: string): Promise<VideoSummary[]> {
115         const data = await request<VideosResponse>("/videos", { method: "GET", cid });
116         return data.videos ?? [];
117     }
118
119     /** The source-video stream URL for annotation playback. GET /api/source/{video_id}
120     (P8). The engine
121     * serves a frame-identical proxy; labels stay keyed to the master's MD5
122     (STUDIO_MEDIA_LIFECYCLE D4). */
123     export function sourceUrl(videoId: string): string {
124         return `${BASE}/source/${encodeURIComponent(videoId)}`;
125     }
126
127     /** Persist finished rally labels as the annotator CSV. POST /api/annotations {video_id,
128     csv} (P8).
129     * Sent as the byte-compatible CSV (serializeRows) so the engine writes it verbatim
130     into the corpus. */
131     export async function postAnnotations(videoId: string, csv: string, cid?: string):
132     Promise<AnnotationsResponse> {
133         return request<AnnotationsResponse>("/annotations", {
134             method: "POST",
135             body: JSON.stringify({ video_id: videoId, csv }),
136             cid,
137         });
138     }
139
140     /** Store a video into the user's chosen medium + register it as a library asset (P4,
141     D14).
142     * POST /api/assets {uploaded_path? | source_uri?, medium_uri, sport?} ? 201 {video_id,
143     stored_uri}.
144     * Exactly ONE source: an `uploadedPath` (a just-uploaded local file ? a durable
145     MD5-keyed copy is
146     * put into the medium, the local upload kept as the working copy) OR a `sourceUri` (a
147     video already
148     * at rest, registered as-is). Feed `stored_uri` into processVideo(). Fast-fails 400
149     (bad medium /
150     * both-or-neither source), 403 (hosted posture), 404 (missing file), 507 (no space),
151     502 (put) ?
152     * surfaced as ApiError. */
153     export async function createAsset(
154         args: { uploadedPath?: string; sourceUri?: string; mediumUri: string; sport?: string
155     },
156         cid?: string,
157     ): Promise<AssetResponse> {
158         return request<AssetResponse>("/assets", {
159             method: "POST",
160             body: JSON.stringify({
161                 uploaded_path: args.uploadedPath,
162                 source_uri: args.sourceUri,
163                 medium_uri: args.mediumUri,
164                 sport: args.sport,
165             }),
166             cid,

```



```

153     });
154 }
155
156 /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
157 export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
158     const data = await request<QueryResponse>("/query", {
159         method: "POST",
160         body: JSON.stringify({ video_id: videoId }),
161         cid,
162     });
163     return data.play_segments ?? [];
164 }
165
166 // Compile-job poll cadence (mirrors useIngestJob): a long ffmpeg stitch must not block
the request.
167 const COMPILE_POLL_MIN_MS = 1000;
168 const COMPILE_POLL_MAX_MS = 5000;
169 const COMPILE_POLL_BACKOFF = 1.5;
170 const COMPILE_POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' compile won't poll
forever
171
172 /** Stitch selected rallies into one reel. ASYNC (#329): POST /api/compile returns 202 +
a job_id;
173 * the ffmpeg stitch runs on the worker and we POLL GET /api/jobs/{id} for the reel ? a
long stitch
174 * would otherwise exceed the ~100 s edge/proxy timeout (524) while the reel actually
built. Resolves
175 * with the job's CompileResult on success; throws ApiError on a failed/timed-out
compile.
176 * `locale` (the UI language) localizes the reel watermark engine-side (omitted when
absent). */
177 export async function compileReel(
178     videoId: string,
179     segmentIds: number[],
180     outputFilename: string,
181     cid?: string,
182     locale?: string,
183 ): Promise<CompileResult> {
184     const c = cid ?? newCorrelationId();
185     const submitted = await request<ProcessResponse>("/compile", {
186         method: "POST",
187         body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename, locale }),
188         cid: c,
189     });
190     return pollCompileJob(submitted.job_id, c);
191 }
192
193 /** Poll a submitted compile job to its terminal state. Returns the reel result on
success; throws an
194 * ApiError for an engine job-failure, a success-shaped FAILURE (done but the stitch
failed / no URL),
195 * or a poll timeout ? so the caller's existing classifyError/errorText mapping
surfaces it like ingest. */
196 async function pollCompileJob(jobId: string, cid: string): Promise<CompileResult> {
197     const startedAt = Date.now();
198     let delay = COMPILE_POLL_MIN_MS;
199     for (;;) {
200         const job = await getJob(jobId, cid);
201         if (job.status === "done") {
202             const result = (job.result ?? {}) as CompileResult;
203             if (result.status === "failed" || !result.download_url) {
204                 // A real stitch failure carries an engine message ? surface it as an `engine`

```

```

detail (status +
205         // body set) so classifyError shows it verbatim, NOT the no-status "engine
unreachable" fallback.
206         const msg = result.message || "Highlight stitching failed.";
207         throw new ApiError(msg, { cid, status: 502, body: msg });
208     }
209     return result;
210 }
211 if (job.status === "failed") {
212     const msg = job.error || "Compile job failed.";
213     throw new ApiError(msg, { cid, status: 502, body: msg });
214 }
215 if (Date.now() - startedAt > COMPILE_POLL_CEILING_MS) {
216     throw new ApiError("Compile timed out.", { cid, status: 504, body: "Compile timed
out." });
217 }
218 await new Promise((resolve) => setTimeout(resolve, delay));
219 delay = Math.min(delay * COMPILE_POLL_BACKOFF, COMPILE_POLL_MAX_MS);
220 }
221 }
222
223 /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
224 export function highlightUrl(videoId: string, filename: string): string {
225     return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
226 }
227
228 /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
229 export async function getConfig(cid?: string): Promise<EngineConfig> {
230     return request<EngineConfig>("/config", { method: "GET", cid });
231 }
232
233 /** What THIS box can do ? drives the first-run wizard. GET /api/capabilities
(PACKAGING_PLAN P2c). */
234 export async function getCapabilities(cid?: string): Promise<EngineCapabilities> {
235     return request<EngineCapabilities>("/capabilities", { method: "GET", cid });
236 }
237
238 /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
239 export function minShotCount(cfg: EngineConfig): number | undefined {
240     return cfg.stitching?.min_shot_count;
241 }
242
243 // ?? chunked upload (backend/api/uploads.py) ?????????????????????????????????
244 // A browser uploads a LOCAL file in sequential parts ? max_part_mb (free-tier edge
proxies cap
245 // bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into
/api/process.
246
247 /** Start a chunked upload. 400 = bad extension; 413 = over the size limit
[uploads.py:64]. */
248 export async function uploadInit(filename: string, totalSizeBytes: number, cid?:
string): Promise<UploadInitResponse> {
249     return request<UploadInitResponse>("/upload/init", {
250         method: "POST",
251         body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
252         cid,
253     });
254 }
255
256 /** Append ONE part (raw bytes ? not JSON). Parts are strictly sequential 0,1,2,?
[uploads.py:89]. */
257 export async function uploadPart(

```

```

258     uploadId: string,
259     index: number,
260     chunk: Blob,
261     cid?: string,
262     signal?: AbortSignal,
263 ): Promise<UploadPartResponse> {
264     const c = cid ?? newCorrelationId();
265     const url = `${BASE}/upload/${encodeURIComponent(uploadId)}/part/${index}`;
266     log("info", "api.request", { cid: c, method: "PUT", url });
267     let res: Response;
268     try {
269         res = await fetch(url, {
270             method: "PUT",
271             body: chunk,
272             headers: { "content-type": "application/octet-stream", "x-correlation-id": c },
273             signal,
274         });
275     } catch (err) {
276         log("error", "api.network_error", { cid: c, method: "PUT", url, error: String(err) });
277         throw new ApiError(`Network error calling PUT ${url}`, { cid: c, cause: err });
278     }
279     if (!res.ok) {
280         const body = await res.text().catch(() => "");
281         log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status,
body: body.slice(0, 500) });
282         throw new ApiError(`PUT ${url} failed: ${res.status}`, { cid: c, status: res.status,
body });
283     }
284     log("info", "api.response", { cid: c, method: "PUT", url, status: res.status });
285     return (await res.json()) as UploadPartResponse;
286 }
287
288 /** Finalize: returns {path, video_id} ? feed `path` to processVideo() [uploads.py:123].
*/
289 export async function uploadComplete(uploadId: string, totalParts: number, cid?:
string): Promise<UploadCompleteResponse> {
290     return
request<UploadCompleteResponse>(`/upload/${encodeURIComponent(uploadId)}/complete`, {
291         method: "POST",
292         body: JSON.stringify({ total_parts: totalParts }),
293         cid,
294     });
295 }
296
297 /** Abort an in-flight upload (best-effort cleanup of the server-side partial)
[uploads.py:147]. */
298 export async function uploadAbort(uploadId: string, cid?: string): Promise<void> {
299     await request<unknown>(`/upload/${encodeURIComponent(uploadId)}`, { method: "DELETE",
cid });
300 }
301
302 export interface UploadProgress {
303     sentBytes: number;
304     totalBytes: number;
305 }
306
307 /** Upload a local File end to end: init ? sequential parts (byte progress, abortable) ?
complete.
308     * On ANY failure or abort it best-effort DELETES the server-side partial, then
rethrows so the
309     * caller can classify it (the engine's 400/413 detail survives). Returns {path,
video_id}.
310     * Note: a per-part 413 (over the per-part limit) aborts the upload server-side ?
recovery is a

```

```

311      * fresh init (re-pick the file), which the UI's retry does. */
312      export async function uploadFile(
313          file: File,
314          opts: { cid?: string; signal?: AbortSignal; onProgress?: (p: UploadProgress) => void }
315      ): Promise<UploadCompleteResponse> {
316          const cid = opts.cid ?? newCorrelationId();
317          const init = await uploadInit(file.name, file.size, cid);
318          // Honor the engine's max_part_mb, but never let a missing/NaN value collapse the math
319          // to NaN (which // would silently upload zero parts) ? fall back to the engine's 95 MB default.
320          const partMb = Number.isFinite(init.max_part_mb) && init.max_part_mb > 0 ?
321          init.max_part_mb : 95;
322          const partBytes = partMb * 1024 * 1024;
323          const totalParts = Math.max(1, Math.ceil(file.size / partBytes));
324          log("info", "upload.start", { cid, filename: file.name, size: file.size, parts:
325          totalParts, part_mb: init.max_part_mb });
326          let sent = 0;
327          try {
328              for (let i = 0; i < totalParts; i++) {
329                  if (opts.signal?.aborted) throw new DOMException("Upload aborted", "AbortError");
330                  const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
331                  await uploadPart(init.upload_id, i, chunk, cid, opts.signal);
332                  sent += chunk.size;
333                  opts.onProgress?.({ sentBytes: sent, totalBytes: file.size });
334              }
335              const done = await uploadComplete(init.upload_id, totalParts, cid);
336              log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size:
337              done.size_bytes });
338              return done;
339          } catch (err) {
340              log("warn", "upload.aborting_partial", { cid, upload_id: init.upload_id, error:
341              String(err) });
342              await uploadAbort(init.upload_id, cid).catch(() => {});
343              throw err;
344          }
345      }
346  }

```

## 9. assistant (2026-07-02T02:47:48.044Z)

Now let me check the engine's API surface to compare against the SPA client.

## 10. user (2026-07-02T02:47:49.626Z)

```

e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
backend:
__pycache__
annotations
api
assets
audio
billing.py
compute
config
detectors
eval
features
flywheel.py
infrastructure
main.py
pipeline
providers

```

reporting  
results.py  
sports  
stitcher  
storage  
tools  
ui  
utils  
worker

backend/api:  
\_\_init\_\_.py  
\_\_pycache\_\_  
auth.py  
job\_worker.py  
static  
uploads.py

### 11. user (2026-07-02T02:47:50.929Z)

```
backend\api\uploads.py:70:@router.post("/api/upload/init")
backend\api\uploads.py:111:@router.put("/api/upload/{upload_id}/part/{index}")
backend\api\uploads.py:157:@router.post("/api/upload/{upload_id}/complete")
backend\api\uploads.py:188:@router.delete("/api/upload/{upload_id}")
backend\main.py:258:@app.get("/", response_class=HTMLResponse)
backend\main.py:379:@app.get("/api/config")
backend\main.py:391:@app.get("/api/health")
backend\main.py:419:@app.get("/api/version")
backend\main.py:429:@app.get("/api/capabilities")
backend\main.py:521:@app.get("/api/videos")
backend\main.py:537:@app.get("/api/reels")
backend\main.py:570:@app.get("/api/reels/{filename}")
backend\main.py:623:@app.get("/api/source/{video_id}")
backend\main.py:663:@app.post("/api/annotations")
backend\main.py:727:@app.post("/api/assets", status_code=201)
backend\main.py:1355:@app.post("/api/process", status_code=202)
backend\main.py:1431:@app.get("/api/jobs/{job_id}/cost")
backend\main.py:1446:@app.get("/api/videos/{video_id}/cost")
backend\main.py:1457:@app.get("/api/jobs/{job_id}")
backend\main.py:1492:@app.get("/api/highlights/{video_id}/{filename}")
backend\main.py:1523:@app.post("/api/query")
backend\main.py:1594:@app.post("/api/compile", status_code=202)
```

### 12. assistant (2026-07-02T02:48:02.661Z)

Endpoints all exist. Now check response-shape drift and the bundle-sync state.

### 13. user (2026-07-02T02:48:03.603Z)

```
1 // Types mirror the engine's verified contract (badminton-highlight-indexer).
2 // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3 // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5 export interface RallyMetadata {
6     /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7     shot_count?: number;
8     /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9     shot_count_confidence?: string;
10    detector?: string;
11    [key: string]: unknown;
12 }
13
14 export interface RallySegment {
```

```

15     /** The integer the UI selects and passes to /api/compile as segment_ids. */
16     id: number;
17     start_time: number;
18     end_time: number;
19     duration: number;
20     /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21     server: string | null;
22     /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23     receiver: string | null;
24     metadata: RallyMetadata;
25     /** motion-only fabricated sub-events ? DO NOT surface. */
26     events?: unknown[];
27 }
28
29 export interface QueryResponse {
30     play_segments: RallySegment[];
31 }
32
33 /** One row of the indexed-video library. GET /api/videos [backend/main.py:447] returns
the most
34 * recent first so the console survives a reload. Every field but `video_id` is
optional/defensive. */
35 export interface VideoSummary {
36     /** The MD5 join key (the contract field). Optional defensively ? an older/regressed
engine may
37     * send only the raw `id`; read it through `lib/video.ts::videoId`, never directly.
*/
38     video_id?: string;
39     /** Raw DB fields an engine may send instead of the enriched contract (resilience
fallbacks). */
40     id?: string;
41     filepath?: string | null;
42     match_name?: string | null;
43     local_path?: string | null;
44     fps?: number | null;
45     resolution?: string | null;
46     created_at?: string | null;
47 }
48
49 export interface VideosResponse {
50     videos: VideoSummary[];
51 }
52
53 /** POST /api/annotations ? persist finished rally labels for a video as the annotator
CSV (P8).
54 * The engine writes the CSV verbatim (byte-compatible with the pipeline), keyed by the
file MD5. */
55 export interface AnnotationsResponse {
56     status: string;
57     num_rows?: number;
58     csv_uri?: string;
59     message?: string;
60 }
61
62 export interface CompileResponse {
63     status: string;
64     /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
65     filepath: string;
66 }
67
68 /** The terminal RESULT of an async reel compile (#329): the job's `result` payload,
surfaced by
69 * GET /api/jobs/{id} once the stitch finishes. `status` is the pipeline verdict
("success" |

```

```

70      * "failed"); a "failed" result carries a `message`. The SPA builds the download URL
with
71      * highlightUrl() (split-origin-safe), but the engine also returns a same-origin
`download_url`. */
72      export interface CompileResult {
73          status: string;
74          filepath?: string;
75          download_url?: string;
76          message?: string;
77          video_id?: string;
78      }
79
80      export interface EngineConfig {
81          stitching?: {
82              /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
*/
83              min_shot_count?: number;
84              [key: string]: unknown;
85          };
86          [key: string]: unknown;
87      }
88
89      /** GET /api/capabilities [backend/main.py] ? an HONEST probe of what THIS box can do,
rendered by
90      * the first-run wizard (PACKAGING_PLAN.md P2c). Presence-bools only (never the Gemini
key value).
91      * Every field optional/defensive: a missing field degrades the wizard gracefully. */
92      export interface EngineCapabilities {
93          api_version?: number;
94          engine_version?: string;
95          segmenters?: string[];
96          default_segmenter?: string;
97          ffmpeg?: { ffmpeg?: boolean; ffprobe?: boolean; ffmpeg_path?: string; ffprobe_path?:
string };
98          gpu?: { nvidia_smi_present?: boolean };
99          /** Whether a Gemini key is PRESENT (never its value) ? drives the AI-highlights step.
*/
100         gemini_key_present?: boolean;
101         wasb_weights_present?: boolean;
102         /** `cloud_available` = can THIS server satisfy a per-request compute="cloud" dispatch
(a Cloud Run
103         * GPU Job)? The ingest compute-picker shows the "in the cloud" option only when
true. */
104         deployment?: { profile?: string; compute_target?: string; cloud_available?: boolean };
105         /** Which storage media THIS box can store INTO (P3). The medium picker renders only
`usable`
106         * backends; `free_gb` is `null` for a cloud bucket (no readable capacity). Absent on
older engines
107         * ? the SPA degrades to local-only (no picker). */
108         storage?: StorageCapabilities;
109         [key: string]: unknown;
110     }
111
112     /** One storable medium reported by GET /api/capabilities (P3). Presence/usable bools
only ? no
113     * secrets. `id`/`kind` are the backend name (local/gcs/s3/gdrive). */
114     export interface StorageBackendInfo {
115         id: string;
116         kind: string;
117         label: string;
118         usable: boolean;
119         /** Free space in whole GB, or `null` when unknowable (a cloud bucket). */
120         free_gb?: number | null;
121     }
122

```

```

123     export interface StorageCapabilities {
124         default_medium?: string;
125         backends?: StorageBackendInfo[];
126     }
127
128     /** POST /api/assets ? 201 (P4). The video was stored into the chosen medium and
registered as a
129     * library asset keyed by the whole-file MD5 (`video_id`). `stored_uri` is the durable
location to
130     * feed into POST /api/process (a bucket/Drive URI, or a local path for "this
computer"). */
131     export interface AssetResponse {
132         video_id: string;
133         /** The medium backend the asset lives in (local | s3 | gcs | gdrive). */
134         medium: string;
135         /** The durable, MD5-keyed location ? pass as video_path to processVideo(). */
136         stored_uri: string;
137         /** True when a durable copy was made (uploaded ? remote medium); false when
registered in place. */
138         copied: boolean;
139         sport?: string | null;
140         status?: string;
141     }
142
143     /** POST /api/process ? 202 [backend/main.py:260]. The job is queued; poll GET
/api/jobs/{job_id}. */
144     export interface ProcessResponse {
145         job_id: string;
146         /** Always "queued" at submit. */
147         status: string;
148         /** Relative poll path, e.g. "/api/jobs/<id>". */
149         poll: string;
150     }
151
152     /** Chunked upload (backend/api/uploads.py). The browser uploads a local file in
sequential parts
153     * ? max_part_mb, then `complete` returns a server-host `path` to feed straight into
POST /api/process. */
154     export interface UploadInitResponse {
155         upload_id: string;
156         /** Per-part ceiling in MB ? slice the file into parts this big. */
157         max_part_mb: number;
158         next_part: string;
159     }
160     export interface UploadPartResponse {
161         upload_id: string;
162         received_parts: number;
163         received_bytes: number;
164     }
165     export interface UploadCompleteResponse {
166         /** Absolute path on the engine host ? pass as video_path to POST /api/process. */
167         path: string;
168         video_id: string;
169         size_bytes: number;
170     }
171
172     /** GET /api/jobs/{job_id} [backend/main.py:303]. Two status layers:
173     * `status` = the EXECUTION state; `result.status` = the pipeline VERDICT
(success|failed).
174     * A `done` job whose `result.status` === "failed" (or with no video_id) is a success-
shaped
175     * FAILURE ? the UI must not treat it as a load. */
176     export interface JobStatus {
177         job_id: string;
178         status: "queued" | "running" | "done" | "failed";

```



```

179      /** Free-text coarse phase ("hashing", "segmenting (gemini)", "finished") ? NOT a
percentage. */
180      progress?: string | null;
181      /** The canonical video_id, surfaced top-level on success (also in result.video_id).
*/
182      video_id?: string | null;
183      /** Execution error (worker crash / unexpected) when status === "failed". */
184      error?: string | null;
185      result?: { status?: string; video_id?: string; message?: string } | null;
186    }
187

```

#### 14. user (2026-07-02T02:48:05.474Z)

```

/c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/static/:
total 52
drwxr-xr-x 1 avidu 197609    0 Jun 25 16:24 .
drwxr-xr-x 1 avidu 197609    0 Jul  2 08:13 ..
-rw-r--r-- 1 avidu 197609 17913 Jun 25 16:24 app.js
-rw-r--r-- 1 avidu 197609  8743 Jun  7 14:14 index.html
-rw-r--r-- 1 avidu 197609 13418 Jun  7 14:14 style.css

```

```

/c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/ui/:
total 18
drwxr-xr-x 1 avidu 197609    0 Jun 25 16:24 .
drwxr-xr-x 1 avidu 197609    0 Jul  2 08:13 ..
-rw-r--r-- 1 avidu 197609  683 Jun 25 16:24 THIRD_PARTY_NOTICES.md
drwxr-xr-x 1 avidu 197609    0 Jun 25 16:24 assets
-rw-r--r-- 1 avidu 197609  401 Jun 25 16:24 index.html
-rw-r--r-- 1 avidu 197609  273 Jun 25 16:24 ui_version.json
index-Jp9DK5o0.css
index-ukuS9wxa.js
noto-sans-devanagari-devanagari-wght-normal-DWI_L5Cu.woff2
noto-sans-devanagari-latin-ext-wght-normal-CLkupaxV.woff2
noto-sans-devanagari-latin-wght-normal-Bxfr4UJH.woff2
noto-sans-kannada-kannada-wght-normal-DuwB-P8d.woff2
noto-sans-kannada-latin-ext-wght-normal-MdBBF41x.woff2
noto-sans-kannada-latin-wght-normal-DeLZCgFT.woff2

```

#### 15. user (2026-07-02T02:48:14.699Z)

```

1    {
2      "spa_repo": "https://github.com/avidullu/khelsutra.git",
3      "spa_commit": "ed33a515a4c68cb42134e8c7d4514df2addb9f54",
4      "engine_api_version": 1,
5      "content_hash": "b0801e7a231d4849cb33d374840f92d7ab2aca3f6ba9088124bbdd674ff15851",
6      "built_at": "2026-06-22T17:59:59Z"
7    }
8

```

#### 16. user (2026-07-02T02:48:16.379Z)

```

f9eea14 2026-07-01 web: resilient video-summary readers + show fps in the library (#140)
fb2f561 2026-07-01 web: persistent "FFmpeg not detected" banner (hard-blocker warning) (#138)
8667dbf 2026-07-01 P5: SPA storage-medium picker (store a video into your medium) (#137)
cc06c32 2026-07-01 feat(web): P8 annotate-in-UI ? AnnotatePanel over the vendored core (flagged)
(#128)
9e9df72 2026-07-01 feat(web): keep the video name visible after open (persistent current-video
label) (#127)
926b53a 2026-07-01 feat(web): vendor the rally-annotator annotation core (P8 groundwork) (#126)
39c1349 2026-07-01 feat(web): P6 ? "Your videos" library (reopen a stored video) (#124)
d0285cd 2026-07-01 test(e2e): single-origin mock engine + Playwright screencast harness (#122)
ed33a51 2026-06-22 feat(build): poll the async /api/compile job for the reel (#329) (#108)
f67dfe7 2026-06-22 feat(ingest): compute-picker ? choose local vs cloud GPU per run (item 3b)

```

```
(#104)
e2570e4 2026-06-22 docs: archive the Friendly & Multilingual tracker (engineering complete)
(#93)
7ae9cee 2026-06-22 feat(web): P2c first-run setup wizard (3 steps from /api/capabilities, 6
languages) (#88)
---
f9eea14 2026-07-01 22:08:00 +0530
```

## 17. user (2026-07-02T02:48:31.279Z)

```
230     # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
an
231     # arbitrary directory finds the shipped assets instead of creating a stray
./backend/api/static.
232     # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
233     _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
234     try:
235         os.makedirs(_STATIC_DIR, exist_ok=True)
236     except OSError:
237         # The dir ships with the package (a no-op create); guard against a read-only install
238         # tree (system-installed Python) since exist_ok suppresses FileExistsError, not
PermissionError.
239         pass
240     app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
241
242     # P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed
snapshot
243     # (produced by scripts/bundle_ui.sh); fall back to the legacy in-engine dashboard only
if it's
244     # absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so
no SPA change
245     # is needed; it is query-param driven (?video=?), so serving index.html at `/` + /assets
is enough
246     # (no client-side path routing ? no history-fallback catch-all, which would also be a
route-order hazard).
247     _BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
248     _UI_DIR = (
249         _BUNDLED_UI_DIR
250         if os.path.isfile(os.path.join(_BUNDLED_UI_DIR, "index.html"))
251         else _STATIC_DIR
252     )
253     _UI_ASSETS_DIR = os.path.join(_UI_DIR, "assets")
254     if os.path.isdir(_UI_ASSETS_DIR):
255         app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")
256
257
258     @app.get("/", response_class=HTMLResponse)
259     def serve_index():
260         index_path = os.path.join(_UI_DIR, "index.html")
261         if os.path.exists(index_path):
262             with open(index_path, "r", encoding="utf-8") as f:
263                 return f.read()
264         return "<h3>Sports Highlight Indexer Server is Running. (No bundled UI found.)</h3>"
265
266
267     def _bundled_ui_version() -> Optional[Dict[str, Any]]:
268         """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is
bundled.
269         Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
guard)."""
270         path = os.path.join(_BUNDLED_UI_DIR, "ui_version.json")
271         try:
272             with open(path, "r", encoding="utf-8") as f:
273                 return json.load(f)
274         except (OSError, ValueError):
```

```

275         return None
276
277
278     # Global variables initialized at startup
279     db_instance: Optional[Database] = None
280     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
281     # P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if
it were
282     # GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
283     _instance_lock: Optional[Any] = None
284
285     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
286     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
287     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
288     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
289     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still

```

## 18. user (2026-07-02T02:48:32.138Z)

```

495         resolution = f"{w}x{h}" if w and h else None
496         return (fps if isinstance(fps, (int, float)) else None), resolution
497     return None, None
498
499
500     def _video_summary(row: Dict[str, Any]) -> Dict[str, Any]:
501         """Map a DB video row to the API/SPA contract. ADDITIVE ? keeps the raw
``id``/``filepath`` (an
502         existing test + any old client rely on them) and adds the fields the SPA library +
the persistent
503         "current video" label actually read: ``video_id`` (the MD5 join key = ``id``), a
friendly
504         ``match_name`` (so an uploaded video shows its filename, not a generic fallback),
``local_path``,
505         ``created_at`` (from ``processed_at``, and ``fps``/``resolution`` (from the stored
ffprobe
506         validation details) so the library cards can show them. Mirrors the mock engine's
shape so mock and
507         real agree."""
508         fp = row.get("filepath") or ""
509         fps, resolution = _video_dims(row.get("validation_results"))
510         return {
511             **row,
512             "video_id": row.get("id"),
513             "match_name": _friendly_video_name(fp),
514             "local_path": fp,
515             "created_at": row.get("processed_at") or row.get("created_at"),
516             "fps": fps,
517             "resolution": resolution,
518         }
519
520
521     @app.get("/api/videos")
522     def list_videos(request: Request, limit: Optional[int] = None):
523         """List indexed videos (most recent first) so the console survives a reload ?
previously only a
524         single-row get existed (PACKAGING_PLAN.md P1). Rows are enriched to the SPA contract
525         (`video_id` + a friendly `match_name` derived from the filename) so the library
shows real
526         names and the persistent "current video" label survives an upload ? see
``_video_summary``."""
527         return {"videos": [_video_summary(v) for v in
request.app.state.db.list_videos(limit=limit)]}

```

```

528
529
530     def _reels_dir(cfg) -> str:
531         return getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
532
533
534     _REEL_EXTS = (".mp4", ".mov")
535
536
537     @app.get("/api/reels")
538     def list_reels(request: Request):
539         """List compiled highlight reels present in the output dir, newest first, with a
download URL.

```

## 19. user (2026-07-02T02:48:43.952Z)

```

README.md
__pycache__
bundle_ui.sh
doctor.py
install.py
nightly_email.py
nightly_reelcount.py
nightly_regression.ps1
nightly_regression.py
register_nightly_task.ps1
run_phase3_eval.py
run_process_and_stitch.py
uninstall.py
verify_compute_path.py
---
test.yml
---
/c/Users/avidu/Projects/khelsutra-guru/khelsutra/deploy:
DEPLOY_ALPHA.md
GPU_VM_SERVING.md
cloudflared.config.example.yml
droplet
stage_from_bucket.sh

/c/Users/avidu/Projects/khelsutra-guru/khelsutra/docs:
ANNOTATE_AND_TRAIN.md
DRY_RUN_CUJ.md
LOCAL_APPLIANCE_ARCHITECTURE.md
MULTILINGUAL_EXPANSION.md
PER_RALLY_SELECTION.md
PORTABILITY.md
README.md
REQUEST_ACCESS_FORM.md
STUDIO_MEDIA_LIFECYCLE.md
archives

```

## 20. user (2026-07-02T02:48:44.685Z)

```

import { defineConfig } from "vitest/config";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

// The SPA makes SAME-ORIGIN relative `/api/*` calls. In the appliance (prod) a reverse
// proxy serves the SPA and proxies `/api` to the engine on 127.0.0.1:8000
// (docs/LOCAL_APPLIANCE_ARCHITECTURE.md). In dev, Vite proxies `/api` to the local engine
// (`python -m backend.main --server`, :8000). Override with VITE_ENGINE_ORIGIN if needed.
export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: {

```

```

    proxy: {
      "/api": { target: "http://localhost:8000", changeOrigin: true },
    },
  },
  test: {
    // Default env is node (the pure lib/parser tests); the i18n render test opts into jsdom via
a    // `// @vitest-environment jsdom` docblock. Match .test.tsx too so render tests are
discovered.
    environment: "node",
    include: ["src/**/*.test.{ts,tsx}"],
    // DOM-env localStorage polyfill (jsdom under Node 26 ships none); node-env files no-op.
    setupFiles: ["src/test/setup.ts"],
    // Coverage floor (CI runs `npm run test:coverage`): a merge can't DROP coverage below these
    // thresholds, so untested new code is caught. `all` counts every source file (not only the
ones a
    // test imported), so deleting a test surfaces here too. Excludes are non-logic files: the
app
    // entry/bootstrap, type-only declarations, and the test harness itself.
    coverage: {
      provider: "v8",
      reporter: ["text", "text-summary"],
      all: true,
      include: ["src/**/*.test.{ts,tsx}"],
      exclude: [
        "src/**/*.test.{ts,tsx}", // the tests
        "src/test/**", // jsdom localStorage shim (test harness, not shipped logic)
        "src/main.tsx", // app entry: mounts <App/> + bundles fonts ? nothing to unit-test
        "src/api/types.ts", // type-only declarations (no runtime code)
        "src/annotator/**", // VENDORED from rally-annotator (MIT) ? fully tested upstream; a
smoke test guards the copy
        "**/*.d.ts",
      ],
      // Floor set just below current (stmts/lines 93.3, branches 88.1, funcs 85.9) ? a real
ratchet
      // that fails CI on a meaningful drop, with a little slack so it doesn't flap. Ratchet
upward as
      // coverage grows. (functions stays a touch looser: App.tsx is the engine-orchestrating
shell,
      // costly to exercise fully in unit tests.)
      thresholds: {
        statements: 92,
        branches: 87,
        functions: 84,
        lines: 92,
      },
    },
  },
});

```

## 21. user (2026-07-02T02:48:53.430Z)

```

1    name: test
2    on:
3      push:
4        branches: [master]
5      pull_request:
6
7    jobs:
8      site:
9        runs-on: ubuntu-latest
10       defaults:
11         run:
12           working-directory: site
13       steps:

```

```

14         - uses: actions/checkout@v7
15         - uses: actions/setup-node@v6
16         with:
17             node-version: "24" # node:sqlite is unflagged; server.test.js boots the VM
host
18             cache: "npm"
19             cache-dependency-path: site/package-lock.json
20         # jsdom is a TEST-ONLY devDep (the behavioral landing-page UX tests run the real
inline
21         # script in a DOM); the Cloudflare Worker + VM-host RUNTIMES stay dependency-free.
`npm test`
22         # runs ALL paths: Worker+D1 logic (index.test.js), the booted VM host
(server.test.js), the
23         # static content contract (assets.test.js), and the UX behaviour (ux.dom.test.js).
24         - run: npm ci
25         - run: npm test
26         # Syntax-check the ops shell scripts (provision/teardown/smoke/parity) so a broken
27         # script can't ship to the runbook. `bash -n` parses without executing.
28         - name: shell scripts parse-check
29         run: for f in scripts/*.sh; do echo "checking $f"; bash -n "$f"; done
30
31     web:
32         # Operator-console SPA (web/): typecheck + unit tests (with a coverage floor) +
production build
33         # gate the merge. `test:coverage` fails if coverage drops below the thresholds in
vite.config.ts,
34         # so a merge can't silently regress test coverage.
35         runs-on: ubuntu-latest
36         defaults:
37             run:
38                 working-directory: web
39         steps:
40             - uses: actions/checkout@v7
41             - uses: actions/setup-node@v6
42             with:
43                 node-version: "24"
44                 cache: "npm"
45                 cache-dependency-path: web/package-lock.json
46             - run: npm ci
47             - run: npm run typecheck
48             - run: npm run test:coverage
49             - run: npm run build
50
51     layout:
52         # Real-browser layout-regression guard. The other suites run in jsdom, which has NO
layout engine
53         # (getBoundingClientRect() is all zeros) and therefore cannot see header overflow /
wrapping /
54         # clipping ? the responsive-nav bug that recurred. This drives a headless Chromium
over the real
55         # pages at a spread of viewport widths x all 6 locales and asserts the header never
distorts.
56         runs-on: ubuntu-latest
57         defaults:
58             run:
59                 working-directory: site
60         steps:
61             - uses: actions/checkout@v7
62             - uses: actions/setup-node@v6
63             with:
64                 node-version: "24"
65                 cache: "npm"
66                 cache-dependency-path: site/package-lock.json
67             - run: npm ci
68             - run: npx playwright install --with-deps chromium

```

```
69         - run: npm run test:layout
70         # Cross-locale render guard: drives the real pages in every locale and asserts the
live DOM
71         # matches the catalog (no raw keys / no shim regression / right lang / no console
errors).
72         - run: npm run test:render
73         # Accessibility guard: axe-core WCAG 2.1 A/AA over every page × representative
locales; fails on
74         # any serious/critical violation (jsdom has no contrast/layout engine to catch
these).
75         - run: npm run test:a11y
76
```

## 22. user (2026-07-02T02:48:54.554Z)

```
deploy/droplet:
README.md
config.json.example
engine.env.example
khelsutra-engine.service
```

```
locales:
da
en
es
hi
id
kn
```

```
site/public:
capture.html
flyer-da.html
flyer-es.html
flyer-hi.html
flyer-id.html
flyer-kn.html
flyer.html
i18n.js
img
index.html
locales
og.png
own-model.html
privacy.html
robots.txt
video
```

```
site/scripts:
copy-locales.mjs
droplet-smoke.sh
provision-vm.sh
teardown-vm.sh
```

```
site/src:
a11y.browser.mjs
assets.test.js
deploy-config.test.js
i18n.shim.test.js
index.js
index.test.js
layout.browser.mjs
locales-drift.test.js
notify.js
render.browser.mjs
server.test.js
```

store.js  
ux.dom.test.js

### 23. user (2026-07-02T02:49:03.300Z)

```
1      # Deploy KhelSutra Studio for alpha ? a shareable page, no SSH
2
3      > **Status:** `LIVE` (deploy live-verified on the droplet 2026-06-22 ?
`deploy/droplet/README.md`; only a rotated Gemini key + a browser OTP login remain to exercise
the upload?reel CUJ) . **Topology:** split ? SPA on Cloudflare Pages + engine on an `api.`
cloudflared tunnel, Cloudflare Access over both (the engine's `HOSTING_PLAN.md` $2 alpha plan).
. **Cost:** ~$0/mo standing (Pages + Tunnel + Access are free ?50 users) + Gemini per-run
(~$0.05/match).
4
5      This turns the local-only flow (run engine + SPA on your box, drive it yourself) into a
**page you can share with alpha testers** ? they open `app.khelsutra.guru`, sign in (email
allowlist), point at a video, and get a reel. No SSH, no home IP exposed, no inbound ports.
6
7      > **Outgrown the droplet?** To move the live engine onto a **GCP L4 GPU VM** (more
CPU/RAM for ffmpeg
8      > + optional WASB GPU detection) and cut the tunnel over to it, see
**[`GPU_VM_SERVING.md`](GPU_VM_SERVING.md)**
9      > ? same tunnel + Access, no DNS change, droplet kept as fallback (~$500?600/mo while
the VM runs).
10
11     ## 0. The shape
12
13     ```
14     tester browser
15     ? https
16     ?
17     Cloudflare edge (free): DNS . TLS . Access (email-OTP allowlist) in front of BOTH
hostnames
18     ??? app.khelsutra.guru ? Cloudflare Pages          (the web/ SPA; built with
VITE_API_BASE)
19     ??? api.khelsutra.guru ? cloudflared Tunnel (outbound-only) ?? 127.0.0.1:8000 on
YOUR box
20
21
22
23
24
25     - **No inbound ports.** `cloudflared` dials out; the engine stays bound to
`127.0.0.1:8000`. The box's home/public IP is never exposed.
26     - **Auth at the edge.** Cloudflare Access gates both hostnames; the engine *also*
verifies the `Cf-Access` JWT in-app (defence in depth ? `security.edge_auth_enabled`), so a
direct hit to the tunnel can't spoof identity.
27     - **Cross-origin, on purpose.** `app.` (SPA) and `api.` (engine) are different origins ?
the SPA is built with `VITE_API_BASE=https://api.khelsutra.guru/api` (PR #64) and the engine
locks CORS to the SPA origin via `RALLY_CORS_ALLOW_ORIGINS` (no engine code change ? it's
already env-driven).
28
29     > **Alternative (single-origin):** if you'd rather run **one box** that serves the SPA
*and* the API behind one origin (Caddy reverse-proxy, no Pages, no cross-origin), see
`LOCAL_APPLIANCE_ARCHITECTURE.md` D1. This runbook ships the **split** topology you chose
(stable Pages-hosted page + a thin API tunnel).
30
31     ---
32
33     ## 1. Prerequisites
34
35     - A **box** to run the engine: the CPU droplet `168.144.126.176` (Gemini-only ? no GPU;
fine for alpha) **or** your Windows+WSL GPU box (real WASB). `ffmpeg` + Python 3.8+ installed;
```



```

the `badminton-highlight-indexer` engine checked out.
36     - A Cloudflare account with the `khelsutra.guru` zone (you already run the marketing
site there).
37     - `cloudflared` installed on the box (https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/downloads/).
38     - A fresh `GEMINI_API_KEY` (rotate the chat-shared one ? it uploads downsized chunks
to Google; this is a paid + cloud action).
39
40     ---
41
42     ## 2. Engine (the `api.` side)
43
44     ### 2.1 Install (with the auth extra)
45     ```bash
46     cd badminton-highlight-indexer
47     pip install -e .[auth] # PyJWT[crypto] ? REQUIRED when edge auth is on, else
every authenticated request is rejected 401
48     ```
49
50     ### 2.2 The safe-exposed config
51     Create / edit `config.json` on the box. Where it must live: the engine reads
`$RALLY_APP_HOME/config.json` if `RALLY_APP_HOME` is set, else `./config.json` relative to the
directory you launch the server from ? not necessarily the repo root. If the engine reads a
different (or absent) file it silently falls back to all-defaults, which means edge auth is
OFF. To make writer and reader agree by construction, write it with python -m backend.main
--setup (it targets the exact path the server reads) and edit that file:
52     ```jsonc
53     {
54         "security": {
55             "edge_auth_enabled": true, // turn the CF-Access JWT verify ON
56             "allowed_video_root": "/srv/khelsutra/incoming", // the path-jail ? REQUIRED when
exposed
57             "upload_dir": "/srv/khelsutra/incoming/uploads", // MUST sit UNDER
allowed_video_root (see §7)
58             "cf_team_domain": "https://<your-team>.cloudflareaccess.com",
59             "cf_access_aud": "<the CF Access application AUD>" // filled in §4
60         },
61         "indexing": {
62             "default_segmenter": "gemini" // ? CPU box MUST set this ? the shipped default
`yolo_hybrid` needs torch/GPU, and the SPA sends no segmenter, so every job fails "No module
named 'torch'" without it
63         }
64     }
65     ```
66     And export the env (a .env` or your shell):
67     ```bash
68     export GEMINI_API_KEY="<your-rotated-key>"
69     export RALLY_CORS_ALLOW_ORIGINS="https://app.khelsutra.guru" # lock CORS to the SPA
origin
70     export RALLY_ALLOWED_HOSTS="api.khelsutra.guru,localhost,127.0.0.1" # REQUIRED ? see ?
below; without it EVERY tunneled request 400s (loopback added so on-box /api/health checks still
pass ? setting this var REPLACES the loopback-only default)
71     export RALLY_BIND_HOST="127.0.0.1" # keep the loopback bind
(cloudflared dials 127.0.0.1:8000)
72     ```
73
74     > Two env vars you MUST set, or the tunnel is dead-on-arrival. With
`edge_auth_enabled: true`, the engine (a) flips its bind to `0.0.0.0` and (b) keeps the DNS-
rebinding Host-header guard at its loopback-only default ? so it boots "healthy" but then
400`s every request cloudflared forwards with Host: api.khelsutra.guru (including the
§6 /api/health` check). The boot only warns about this (EXPOSED_HOST_ALLOWLIST_LOOPBACK`),
it does not stop. Set `RALLY_ALLOWED_HOSTS="api.khelsutra.guru,localhost,127.0.0.1"`` (the
load-bearing fix ? the Host allowlist is independent of the bind; the loopback entries keep the
on-box /api/health` check passing, since setting this var REPLACES the loopback-only default)
and `RALLY_BIND_HOST="127.0.0.1"`` (so the bind stays loopback as §0/§6 claim; cloudflared`

```

```

dials loopback anyway). *Verified against engine `origin/master`: with the allowlist unset, `GET
/api/health` with `Host: api.khelsutra.guru` ? `400`; with it set ? `200`.*
75
76 > **The engine refuses to start unsafe.** The boot posture (engine PR #294) **fails
fast** if `edge_auth_enabled` is on but the `allowed_video_root` jail or
`cf_team_domain`/`cf_access_aud` is missing, and **warns** if `PyJWT` isn't installed. So a
half-configured exposure can't silently come up ? fix what it prints and restart.
77
78 ### 2.3 Run
79 ```bash
80 python -m backend.main --server # binds 127.0.0.1:8000 ? kept loopback by
RALLY_BIND_HOST ($2.2); with edge_auth ON and that var unset it would bind 0.0.0.0 instead
81 ```
82 Keep it alive with a process manager (`systemd` on the droplet, mirroring
`site/scripts/provision-vm.sh`'s unit; or `pm2`/`nssm` on Windows).
83
84 ---
85
86 ## 3. The SPA on Cloudflare Pages (the `app.` side)
87
88 The SPA ships as a **static Pages site**, git-connected like the marketing site (auto-
deploy on merge to `master`).
89
90 1. **Cloudflare dashboard ? Workers & Pages ? Create ? Pages ? Connect to Git** ? pick
the `khelsutra` repo.
91 2. **Build settings:**
92 - **Root directory:** `web`
93 - **Build command:** `npm run build`
94 - **Build output directory:** `web/dist`
95 - **Environment variable:** `VITE_API_BASE = https://api.khelsutra.guru/api` ? this
is what points the SPA at the engine tunnel (PR #64).
96 3. **Custom domain:** add `app.khelsutra.guru` to the Pages project.
97
98 > Every push to `master` now rebuilds + redeploys the SPA. To preview a change first,
Pages builds a per-PR preview URL.
99
100 ---
101
102 ## 4. Cloudflare Access (gate BOTH hostnames)
103
104 1. **Zero Trust ? Access ? Applications ? Add ? Self-hosted.**
105 2. **Application domain:** add **both** `app.khelsutra.guru` **and**
`api.khelsutra.guru` to the *same* application (one app ? one shared session cookie, so a tester
signs in once and both the SPA and its API calls are authorized).
106 3. **Policy:** Action *Allow*, rule *Emails* ? your alpha testers' addresses (free ? 50
users ? fits 10?25 testers). Use one-time-PIN (email OTP) so testers need no Cloudflare account.
107 4. **Copy the application `AUD` tag** ? paste into the engine's `config.json`
`security.cf_access_aud` ($2.2), and set `cf_team_domain` to `https://<your-
team>.cloudflareaccess.com`. Restart the engine.
108 5. **CORS preflight ?:** the browser sends an unauthenticated `OPTIONS` preflight to
`api.` for the cross-origin POSTs. In the Access application's **Settings ? enable ?Bypass
options requests to CORS preflight?** (or add an explicit OPTIONS bypass), else the preflight is
redirected to the login page and every API call fails. The engine's CORS middleware
(`allow_credentials=false`, no cookies) then answers the preflight.
109
110 ---
111
112 ## 5. The tunnel (engine ? Cloudflare)
113
114 ```bash
115 cloudflared tunnel login # browser auth to your zone
116 cloudflared tunnel create khelsutra-studio # note the <TUNNEL_UUID>
117 # copy deploy/cloudflared.config.example.yml ? /etc/cloudflared/config.yml, fill
<TUNNEL_UUID>
118 cloudflared tunnel route dns khelsutra-studio api.khelsutra.guru

```

```

119     cloudflared tunnel run khelsutra-studio                # or install as a service:
`cloudflared service install`
120     ```
121     See `deploy/cloudflared.config.example.yml` for the ingress (only `api.khelsutra.guru` ?
http://127.0.0.1:8000`).
122
123     ---
124
125     ## 6. Verify (the posture test + the CUJ)
126
127     **Exposure posture (must hold ? this is what makes it safe to share):**
128     - From OUTSIDE the box: `curl --max-time 5 http://<box-public-ip>:8000/api/health` ?
**refused / timeout** (the engine is `127.0.0.1`-bound; no inbound port). ? The box is not
directly reachable.
129     - `curl -i https://api.khelsutra.guru/api/health` **without** a CF session ? a **302 to
the Access login** (not a 200). ? The gate is in front.
130     - Open `https://app.khelsutra.guru` in a browser ? CF Access login ? after sign-in, the
SPA loads and `/api/health` returns 200 (same session cookie). ?
131
132     **The shareable CUJ (what a tester does):** open `app.khelsutra.guru` ? sign in ?
**upload a video from their device** (the SPA chunked-upload lands it in `upload_dir`, inside
the jail) ? **Process** ? watch progress ? pick rallies ? **Build** ? **download the reel**. All
from the page. *(Pasting a `gs://` URI does NOT work on this box ? see §7: the
`allowed_video_root` jail rejects every non-local URI. Browser upload is the alpha path.)*
133
134     ---
135
136     ## 7. Honest caveats / deferred
137
138     - **`upload_dir` must sit UNDER `allowed_video_root`.** If you enable browser upload, an
upload landing outside the jail is rejected by `/api/process` (a functional break, not a hole).
Keep `upload_dir` inside the jail (§2.2).
139     - **`gs://` / `s3://` / `gdrive://` URIs are REJECTED on the exposed box.** Setting
`allowed_video_root` (required to expose, §2.2) path-jails `/api/process` to *local* paths under
that root; the engine resolves a `gs://` URI to a local path that escapes the jail ? `400`
(pinned by engine `tests/test_api_endpoints.py::test_process_hosted_mode_rejects_nonlocal_uri`).
Cloud-URI ingest works **only** when `allowed_video_root` is unset (the local single-user
posture ? the opposite of this runbook). For alpha, the ingest path is **browser upload** (it
lands inside the jail). *(Scheme-exempting recognized buckets from the jail would be an engine
change, not a config flip.)*
140     - **You MUST set `edge_auth_enabled=true` to expose.** The boot posture (and the in-app
JWT verify) only engage then. Do not put the engine behind the tunnel with edge auth off.
141     - **Cross-origin reel download:** the reel streams from `api.` with `Content-
Disposition: attachment`, so it downloads even cross-origin ? but verify the filename in your
browser during the CUJ (cross-origin `` filename hints are ignored; the header is
the authority).
142     - **Upload size:** Cloudflare's free proxy caps request bodies at **100 MB** ? large
matches upload as ~95 MB chunks (the SPA chunked-upload path honours the engine's `max_part_mb`,
default **95** ; lower `security.max_part_mb` to shrink them). `gs://` bucket pre-staging is
**not** an option on this box (see the jail caveat above).
143     - **Box offline = engine offline.** The Pages SPA stays up and shows a friendly "engine
offline" banner (`/api/health` ping); processing resumes when the box is back. Beta removes this
with a queue (`HOSTING_PLAN.md` §3).
144     - **Gemini = paid + uploads footage to Google.** Keep the key rotated; use it only for
clips you're OK sending.
145
146     ---
147
148     ## 8. Teardown
149     `cloudflared tunnel cleanup khelsutra-studio` + delete the tunnel; remove the Pages
custom domain; disable the Access application; stop the engine service. The box's `output/` +
`uploads/` retain originals ? sweep them per your retention policy before any wider exposure.
150
151     ---
152     ### Changelog

```

153 - 2026-06-22 ? **\*\*correctness pass\*\*** (adversarial verify against engine `origin/master` + a hermetic repro). Fixed two would-break-the-deploy gaps: (1) **\*\*`RALLY\_ALLOWED\_HOSTS`** was missing\*\* ? with `edge\_auth` on, the Host-header guard 400s every tunneled request until it's set (added it + `RALLY\_BIND\_HOST` to \$2.2; repro confirmed 400?200); (2) **\*\*`gs://`** ingest is rejected\*\* by the `allowed\_video\_root` jail on the exposed box (dropped the `gs://` promise from \$6/\$7; alpha path = browser upload). Minor: 401-not-500 wording (\$2.1), `config.json` location/`--setup` note (\$2.2), bind comment (\$2.3), ~95 MB chunk size (\$7).

154 - 2026-06-21 ? created. Split-topology alpha deploy kit (cloudflared + Pages + CF Access) consuming the engine exposure-safety boot posture (engine #294) + the SPA `VITE\_API\_BASE` (#64). Pairs with `LOCAL\_APPLIANCE\_ARCHITECTURE.md` (single-origin alternative) + the engine `HOSTING\_PLAN.md` \$2.

155

**24. user (2026-07-02T02:49:05.008Z)**

## Move KhelSutra Studio onto an L4 GPU VM (off the droplet)

Self-serve runbook to run the live engine (behind `api./app.khelsutra.guru`) on a **\*\*GCP L4 GPU VM\*\*** instead of the CPU droplet, and **\*\*cut the Cloudflare tunnel over\*\*** so traffic lands on the VM. Lets you switch detection between **\*\*Gemini\*\*** (cloud API, proven) and **\*\*WASB native\*\*** (uses the L4 GPU).

This extends [`DEPLOY_ALPHA.md`](DEPLOY\_ALPHA.md) (which deployed the same engine on the droplet) ? it reuses the droplet config templates in [`deploy/droplet/`](droplet/) and the GPU-VM scripts in the **\*\*engine repo\*\*** (`badminton-highlight-indexer`:` deploy/gcp/``).

> **\*\*These are manual steps today.\*\*** Automating them with a one-shot **\*\*startup-script\*\*** you feed into  
> the GCP console (`create_gpu_vm.sh`` already injects a metadata startup-script ? today it only installs  
> the Ops Agent) is backlogged: engine repo `docs/BACKLOG.md`` ? **\*Batch / declarative orchestration\***. The  
> design hinge there is keeping the secrets (Gemini key / Access AUD / tunnel token) out of plaintext  
> metadata.

> **\*\*Cost ??\*\*** A `g2-standard-4`` (1xL4) bills **\*\*~\$0.70?0.85/hr ? ~\$500?600/month\*\*** while it runs. You  
> keep it up, so this is an ongoing cost. **\*\*`bash deploy/gcp/teardown.sh delete`** is the only way to  
> stop billing\*\* (a **\*stopped\*** VM still bills its disk). For bursty/intermittent load, a scale-to-zero  
> **\*\*Cloud Run GPU \*service\*\*\*** would be far cheaper (~\$0 idle) ? that's the backlogged "M7" and a bigger  
> build; this VM is the fast path that works today.  
>  
> **\*\*Cutover is safe + reversible.\*\*** Bring the VM up and **\*\*verify it serving\*\*** before touching the  
> tunnel; the droplet stays as a hot fallback. The tunnel is **\*\*token-managed\*\*** (ingress lives in the  
> Cloudflare dashboard, not on the box), so moving it is just "run the same connector on the VM, then  
> stop it on the droplet" ? **\*\*no DNS change, no new Cloudflare token, no CF-Access change.\*\***

---

## 0. Prerequisites (gather these once)

- **\*\*gcloud\*\*** authenticated as the project owner (`gcloud auth login``) on the machine you run this from.

- **GPU quota** `GPUS\_ALL\_REGIONS ? 1` and the SA Ops-Agent roles ? see the engine repo `deploy/gcp/README.md` S2. (You already have L4 quota=1 from the nightly runs.)
- **The two secrets**, which you already have from the droplet deploy:
  - the **rotated Gemini API key** (Drive: `gemini.api\_key.txt`) ? only needed for the Gemini path;
  - the **CF-Access AUD** + team domain ? already in the droplet's `/srv/khelsutra/config.json` (`security.cf\_access\_aud` / `cf\_team\_domain`); reuse the same values.
  - the **tunnel token** ? in the droplet's cloudflared unit (`systemctl cat cloudflared` ? the `--token` ? value). You'll copy it to the VM in S6.

====DROPLET-README====

## Droplet quickstart ? KhelSutra alpha (the `api.` engine side)

The **filled-in, command-dense** version of `../DEPLOY\_ALPHA.md` for the Linux CPU droplet (`168.144.126.176`, Gemini, **no GPU**). Values are pre-baked for **app.khelsutra.guru` / `api.khelsutra.guru`**, team domain **https://khelsutra.cloudflareaccess.com**, allowlist **avi.dullu@gmail.com**, jail **/srv/khelsutra/incoming**. Read `../DEPLOY\_ALPHA.md` for the **why**;

this is the **what to type**. Files here:

| File                                | Goes to                     | Purpose                                                    |
|-------------------------------------|-----------------------------|------------------------------------------------------------|
| `config.json.example`               | /srv/khelsutra/config.json  | the `security` block (edge-auth + jail + CF team/AUD)      |
| `engine.env.example`                | /etc/khelsutra/engine.env   | secrets + the <b>two `RALLY_`</b> exposure vars + app-home |
| `khelsutra-engine.service`          | /etc/systemd/system/        | keeps the engine up (binds `127.0.0.1:8000`)               |
| `../cloudflared.config.example.yml` | /etc/cloudflared/config.yml | the tunnel ingress (`api.` ? `127.0.0.1:8000`)             |

> **Cost reality:** Cloudflare Pages + Tunnel + Access are **free** (Access ? 50 users). **No \$25/Pro**

> **plan.** When you enable Zero Trust you may be asked to add a card and "subscribe" ? pick the **Free**

> **plan;** you won't be charged under 50 users. Standing cost ? \$0 + Gemini per run.

---

### A. Engine on the box (one-time)

```
```bash
```

#### 1. user + jail dirs (upload\_dir MUST sit under allowed\_video\_root)

```
sudo useradd --system --create-home --home-dir /opt/khelsutra khelsutra || true
sudo mkdir -p /srv/khelsutra/incoming/uploads
sudo chown -R khelsutra:khelsutra /srv/khelsutra /opt/khelsutra
```

#### 2. engine code + venv + the AUTH extra (REQUIRED with edge auth on, else every auth)

25. user (2026-07-02T02:49:17.418Z)

No matches found

26. user (2026-07-02T02:49:17.886Z)

Found 11 files

```
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\client.test.ts
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\client.ts
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\docs\DRY_RUN_CUJ.md
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\docs\LOCAL_APPLIANCE_ARCHITECTURE.md
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\NEXT_STEPS.md
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\deploy\GPU_VM_SERVING.md
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\deploy\DEPLOY_ALPHA.md
```

C:\Users\avidu\Projects\khelsutra-guru\khelsutra\deploy\droplet\engine.env.example  
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\deploy\droplet\README.md  
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\vite-env.d.ts  
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\deploy\cloudflared.config.example.yml

## 27. user (2026-07-02T02:49:24.031Z)

```
1      # Next steps (live) ? KhelSutra brand / site
2
3      Prioritized "what to do next" for the `khelsutra` product repo2. Refresh at each
checkpoint.
4
5      **Where we are (updated 2026-07-01):** `khelsutra.guru` is **LIVE and fully shipped2 ?
landing page + `capture`
6      checklist + OG image + a **segmented Request-Access form** (academy/individual + GPU +
storage + free
7      text). On **Cloudflare** (Worker `hidden-king-khelsutra` + D1 `khelsutra-access`): the
form persists to
8      D1 **and emails a summary to the owner on every registration** (Email Routing send
binding ? verified
9      live). **Git auto-deploy is connected** (Workers Builds, root `site/`): merge to
`master` ? deploys;
10     **CI gates every PR** ? three jobs: the full site `npm test` (Worker + VM host + assets
+ UX), the
11     `web/` SPA build + coverage floor, and a real-browser layout check. The same form also
runs on a
12     dependency-free **Node + SQLite** host (`site/server.js`), **live + browsable on the D0
droplet ?
13     http://168.144.126.176:8787** (systemd `khelsutra-site`). Shipped across ~90 merged PRs
(the form,
14     6-language localization, the per-rally MVP, droplet portability). Docs:
15     `docs/REQUEST_ACCESS_FORM.md`, `docs/PORTABILITY.md`.
16
17     ## Next product threads (owner ideas, 2026-06-20)
18     - **? DONE ? Friendly & Multilingual (ARCHIVED 2026-06-22; human gates cleared):** the
product is
19     non-tech-friendly and **fully localized to all 6 languages** (en/kn/hi/es/da/id)
across both surfaces
20     (`web/` SPA + `site/` marketing) ? including the home page (hero, all sections, the
`#join` access form,
21     footer), the **Privacy Policy** page, **per-page ``s**</b>, and <b>**es/da/id academy</b>
<b>flyers**</b> (with
22     localized SPA screenshots). The responsive header is fixed and a <b>**headless-Chromium</b>
CI suite<sup>2</sup> guards
23     layout, cross-locale render (live DOM == catalog), and <b>**accessibility (axe-core WCAG</b>
2.1 AA<sup>2</sup><b>**</b>. Full
24     record: <b>**`docs/archives/FRIENDLY_AND_MULTILINGUAL.md`**</b>. Post-archive PRs: #86 (own-
model) . #87 (es/da/id
25     site) . #105 (form+footer + render guard) . #106 (Privacy page) . #107 (a11y fixes +
axe guard + page
26     titles) . #109 (flyers). <b>**Owner gates CLEARED:**</b> native-speaker review of the AI-
drafted catalogs ? +
27     real-device font-glyph eyeball ?. <b>**Remaining (small/owner):**</b>
28     - <b>**Delta native-review**</b> of the strings ADDED after the owner's sign-off
(#105/#106/#107/#109 ? the
29     form/footer/privacy/title/flyer copy is AI-drafted): a small top-up review before a
wide public launch.
30     - <b>**B7 (deferred, not a v1 blocker):**</b> engine backend customer-facing l10n (report
<b>`customer_`</b> +
31     rejection reasons) ? cross-repo Python (<b>`badminton-highlight-indexer`</b>), owner-gated
<b>**D-BACKEND-DEPTH**</b>.
32     - <b>**Marketing video on the site**</b> ? embed a short walkthrough of the record ? index ?
reel process,
33     themed for <b>**serious enthusiasts**</b>, on the landing page. ? <b>**A real-UI CUJ screencast</b>
now exists<sup>2</sup></pre></div>
```

```

34      (Playwright recording of the per-rally SPA driven against real GPU-run data ?
`promo_cuj.mp4`/`.gif`
35      in `gs://kshelsutra-rally-corpus/promo/`). Decide: use it as the marketing clip (?
shows real court
36      footage + a dev "P5" badge ? clean up before public embed), and/or produce a polished
Grok/Gemini cut.
37      - **Interactive per-rally selection** (big product feature) ? ? **MVP / M0 COMPLETE**
(the full
38      pick ? query ? select ? compile ? download loop on existing engine endpoints). Lives
in the alpha SPA
39      (`web/`). **? Tracked design + P-tracker (source of truth):
`docs/PER_RALLY_SELECTION.md` (IN PROGRESS;
40      owner-approved, D8?D11 locked). ** **Done: P0 design (#11), P1 scaffold (#14), P2
grid+hook (#21), P3
41      `parseQuery()` query bar (#28), P4 SelectionFooter + `min_shot_count` warning (#30),
P5 real load +
42      Build?compile?preview?download (#31). ** Verified e2e vs a stub engine + a CUJ replay
against the real
43      L4-run rallies + reel. **NEXT (owner-gated, engine repo): M1?M5** ? `GET /api/health`
+ `download_url`
44      (P6), RESTful `?/rallies` alias (P7), multi-video home + auth (P8), per-rally
preview/timeline (P9),
45      corpus correction loop (P10), localized NL (M5). **Optional:** a fresh live
`/api/process` GPU
46      inference once the engine session's M2 training frees the single GPU.
47
48      ## Queued (site polish)
49      - **Always-Use-HTTPS + HSTS** (Cloudflare SSL/TLS) ? closes the plain-HTTP flag.
50      - **Cloudflare Web Analytics (owner-gated; FREE)** ? code side DONE (the dead
placeholder beacon was
51      removed from `site/public/index.html`; analytics is now via **Automatic Setup**, no
in-source token).
52      **Owner step (needs CF login):** the site is a Cloudflare **Worker** (not Pages) on
the proxied
53      `kshelsutra.guru` zone ? enable via **Analytics & Logs ? Web Analytics ? Add a site ?
`kshelsutra.guru`**
54      (Automatic Setup is default-on for proxied hostnames; edge-injects the beacon on every
page). Then
55      filter the view to `hostname = kshelsutra.guru` to exclude `*.pages.dev`/preview noise,
and decide
56      separately whether `app.kshelsutra.guru` (the SPA) needs its own enablement (history-
router only).
57      **Verify (two gotchas):** the Worker's asset responses must NOT send `Cache-Control:
?, no-transform`
58      (Cloudflare can't rewrite that ? no injection), and after a deploy the rendered HTML
should contain the
59      edge-injected beacon. **DONE WHEN:** the Web Analytics dashboard shows incrementing
page views for
60      `kshelsutra.guru`.
61      - **Droplet ? production** ? TLS + a subdomain (`vm.kshelsutra.guru` via Caddy) if it
should be a real
62      standby; add **VM-path SMTP** so the Node host also emails (currently `noopNotify`).
63      - **DMARC** TXT for deliverability; delete any orphan `kshelsutra-site` Worker; clean CF
D1 test rows
64      (`deploy-test@`, `email-test@`). Revoke the droplet SSH key
(`~/ssh/kshelsutra_droplet`) when done.
65
66      ## Bigger arcs (design-first)
67      - **? Studio media lifecycle (ingest ? store ? annotate ? train)** ? **Tracked project:
`docs/STUDIO_MEDIA_LIFECYCLE.md`
68      (IN PROGRESS ? storage half COMPLETE 2026-07-01). ** The seamless flow: add a video ?
store it in a user-chosen medium
69      (S3/GCS/Drive/local) ? free-space checked ? annotate in-UI ? contribute to the
segmenter. **SHIPPED 2026-07-01:**
70      free-space guards (vault + engine, P1/P2), the `/api/capabilities` storage block (P3),

```

```

`POST /api/assets`
71     store-into-medium (P4), the SPA medium picker (P5), the "my videos" library (P6), and
in-UI annotation (P7?P9).
72     **Remaining = owner-gated only:** P10 (corpus promotion) + P11 (train/re-eval ?F1) +
the `L` beta follow-ups.
73     $7 tracker is the source of truth.
74     - **Single-box appliance (Khelsutra Studio)** ? **? Tracked design:
`docs/LOCAL_APPLIANCE_ARCHITECTURE.md` (DRAFT, owner-gated).**
75     The `web/` SPA drives the full **ingest (Drive / bucket URI / local upload) ? index ?
pick rallies ? reel ? download**
76     loop on ONE box, behind an **edge auth gate** (engine stays `127.0.0.1`, CORS `` is
moot). It is the single-box
77     profile of the engine's decoupled `StorageBackend` + `backend.worker` architecture.
MVP runs on the CPU droplet via
78     **Gemini/stub** ; **real local-GPU detection needs a GPU host** (DO droplet / GCE VM /
Cloud Run+GPU ? owner priority
79     ASAP, runs parallel to the UI MVP). Per-rally selection = one screen. Owner decisions
pending in $10 (edge auth,
80     retention, cost ceiling, GPU host, P5 bridge).
81     - **Alpha SPA** in `web/` (Vite + React + TS + Tailwind) ? hosts the operator console +
the per-rally selection UI above.
82     - **BYO-storage + autosync** ? connect Drive/OneDrive/GCS/S3, watch the folder, auto-
prepare reels
83     (engine `PLATFORM_ARCHITECTURE.md $3`). The form's GPU/storage answers are the demand
signal.
84     - **Control plane** (`control-plane/`) at beta.
85
86     ## Org & repos
87     - ? **DONE (2026-06-20): proprietary repos consolidated into the `Khelsutra` org
(private).** ** Transferred
88     `badminton-highlight-indexer` (engine), `sports-data-collector`, `sports-obsreport` ?
`Khelsutra/` (stay private; no
89     open PRs/forks; GitHub redirects git ops so active sessions kept working; local
remotes repointed + verified).
90     `rally-annotator` stays in the org **PUBLIC** (OSS MIT plugin the site links to ? do
not make private). Owner is org
91     admin (full access) and can **invite a contributor** (org member / per-repo outside
collaborator) anytime.
92     - ? **`khelsutra` (this site repo) ? owner step pending.** A transfer breaks the
**Cloudflare Workers-Builds** git
93     connection (auto-deploy) until the Cloudflare GitHub App is installed on the
`Khelsutra` org and the Worker
94     reconnected (dashboard). When ready: install the CF app on the org ? `gh api
repos/avidullu/khelsutra/transfer -f
95     new_owner=Khelsutra` ? reconnect the Worker to `Khelsutra/khelsutra` ? verify a
deploy.
96     - ? **Engine CI under the org** ? verify the engine's GitHub Actions run on the next PR
(free orgs default Actions-on;
97     enable in org settings if a run doesn't start).
98
99     ## Convention
100     Substantial work gets a tracked doc in `docs/`; one PR per item off `origin/master`;
merge to `master`
101     auto-deploys (CI must be green).
102

```

## 28. user (2026-07-02T02:49:34.469Z)

```

deploy\digitalocean\README.md:8:> DO-specific *provisioning* (doctl droplet create, Spaces) is
retired. (Optional later cleanup: move
docs\archives\decisions\DECISIONS.md:681:- The shared DO CPU droplet **`claude-do-rally-test`**
(another session's testbed) is **owner-coordinated** cleanup, separate from this decision; the
freshly-rotated **DO API token** can be retired once that droplet is gone.
docs\archives\past_projects\DECOUPLED_COMPUTE\DEPLOY_REPORT_GCP_2026-06-20.md:99:token was
rotated; with **DO dropped for compute (ADR-013)** it can be retired once the shared DO droplet

```



is gone.

## 29. user (2026-07-02T02:49:35.951Z)

```
// DRIFT GUARD for the one build step the otherwise no-build site has (tracker B3 / D-CATALOG-HOME):
// the catalogs the marketing site ships (site/public/locales/) MUST be byte-identical to the
single
// source of truth (repo-root /locales/). If someone edits /locales without re-running the copy,
this
// fails CI with the exact fix. Run `npm run copy:locales` to resync.
import { test } from "node:test";
import assert from "node:assert/strict";
import { readFileSync, readdirSync } from "node:fs";
import { join } from "node:path";

const SRC = join(import.meta.dirname, "..", "..", "locales");
const PUB = join(import.meta.dirname, "..", "public", "locales");

test("site/public/locales mirrors repo-root /locales exactly (else: npm run copy:locales)", ()
=> {
  const locales = readdirSync(SRC, { withFileTypes: true }).filter((d =>
d.isDirectory()).map((d => d.name);
  assert.ok(locales.length >= 3, "expected at least en/kn/hi catalogs in /locales");

  const shipped = new Set(readdirSync(PUB, { withFileTypes: true }).filter((d =>
d.isDirectory()).map((d => d.name)));
  for (const loc of locales) {
    const src = readFileSync(join(SRC, loc, "common.json"), "utf8");
    assert.ok(shipped.has(loc), `site/public/locales/${loc} is missing ? run npm run
copy:locales`);
    const pub = readFileSync(join(PUB, loc, "common.json"), "utf8");
    assert.equal(pub, src, `site/public/locales/${loc}/common.json drifted from /locales/${loc}
? run npm run copy:locales`);
  }
  // No stale extra locales shipped that aren't in the source of truth.
  for (const loc of shipped) {
    assert.ok(locales.includes(loc), `site/public/locales/${loc} has no source in /locales ? run
npm run copy:locales`);
  }
});
====
locales/en:
common.json

site/public/locales:
da
en
es
hi
id
kn
```

## 30. user (2026-07-02T02:49:47.978Z)

```
en total=473 missing=0 extra=0
kn total=473 missing=0 extra=0
hi total=473 missing=0 extra=0
es total=473 missing=0 extra=0
da total=473 missing=0 extra=0
id total=473 missing=0 extra=0
```

## 31. user (2026-07-02T02:49:48.260Z)

```

// Content regression guards for the localization catalogs ? the rules the adversarial
translation
// audit (#77) established, pinned so the specific bugs can never silently return. This
complements
// hygiene.test.ts (which proves key/placeholder PARITY) by asserting catalog CONTENT. Each
block maps
// to a concrete #77 bug class; the mutation that reintroduces the bug fails here. Pure data
assertions
// over repo-root /locales (D-CATALOG-HOME) ? no render context needed.
import { describe, it, expect } from "vitest";
import { readFileSync, readdirSync } from "node:fs";
import { join } from "node:path";

const LOCALES_DIR = join(import.meta.dirname, "..", "..", "..", "locales");

type Flat = Record<string, string>;
function flatEntries(obj: Record<string, unknown>, prefix = ""): Flat {
  const out: Flat = {};
  for (const [k, v] of Object.entries(obj)) {
    const key = prefix ? `${prefix}.${k}` : k;
    if (v && typeof v === "object" && !Array.isArray(v)) Object.assign(out, flatEntries(v as
Record<string, unknown>, key));
    else out[key] = String(v);
  }
  return out;
}
const load = (loc: string): Flat =>
  flatEntries(JSON.parse(readFileSync(join(LOCALES_DIR, loc, "common.json"), "utf8")) as
Record<string, unknown>);

const ALL_LOCALES = readdirSync(LOCALES_DIR, { withFileTypes: true })
  .filter((d) => d.isDirectory())
  .map((d) => d.name);
const en = load("en");

// (1) Internal-token / debug-tag leaks ? EVERY locale. The engine's internal `min_shot_count`
config
// name leaked into footer.floorSomeRest (#77, kn even wrapped it in <code>). No developer/debug
tag
// (kbd/samp/tt/var) or that token may reach any user-facing string. <code> is likewise banned ?
// EXCEPT on the marketing site's developer "own your model" page (site.ownModel.*), which
intentionally
// renders inline <code> (the rally_annotator.lua filename, the ending_reason enum,
<video>.rallies.csv)
// through the site shim's data-i18n-html form. That namespace is the sole, deliberate
exception; the
// SPA console keys (where the #77 leak happened) still fail on any <code>.
describe("catalog content ? no internal token / debug-tag leaks (every locale)", () => {
  const TOKEN = /min_shot_count/;
  const DEBUG_TAG = /<\/?(?:kbd|samp|tt|var)\b/i;
  const CODE_TAG = /<\/?code\b/i;
  const codeAllowed = (key: string) => key.startsWith("site.ownModel.");
  for (const loc of ALL_LOCALES) {
    it(`${loc}: no 'min_shot_count' token or stray <code>/<kbd>/<samp>/<tt>/<var> tag`, () => {
      const offenders = Object.entries(load(loc))
        .filter(([k, v]) => TOKEN.test(v) || DEBUG_TAG.test(v) || (CODE_TAG.test(v) &&
!codeAllowed(k)))
        .map(([k]) => k);
      expect(offenders).toEqual([]);
    });
  }
});
catalog-invariants.test.ts
hygiene.test.ts
i18n.test.tsx

```

index.ts

### 32. user (2026-07-02T02:50:07.227Z)

```
Exit code 1
site/README.md
site/make_og.py
site/package-lock.json
site/package.json
site/public/capture.html
site/public/flyer-da.html
site/public/flyer-es.html
site/public/flyer-hi.html
site/public/flyer-id.html
site/public/flyer-kn.html
site/public/flyer.html
site/public/i18n.js
site/public/img/annotate-enable.png
site/public/img/annotate-tool.png
site/public/img/flyer-qr.svg
site/public/img/studio-da.png
site/public/img/studio-en.png
site/public/img/studio-es.png
site/public/img/studio-hi.png
site/public/img/studio-id.png
site/public/img/studio-kn.png
site/public/index.html
site/public/locales/da/common.json
site/public/locales/en/common.json
site/public/locales/es/common.json
site/public/locales/hi/common.json
site/public/locales/id/common.json
site/public/locales/kn/common.json
site/public/og.png
site/public/own-model.html
====
site/access.db
check-ignore exit: 0
-rw-r--r-- 1 avidu 197609 12288 Jun 20 20:15 site/access.db
node_modules/
dist/
.env
.env.*
*.local
.DS_Store
Thumbs.db
vm-state-backups/
```

**local runtime / machine-specific ? never commit**

**server.js creates site/access.db from schema.sql (idempotent) ? runtime only**

\*.db

**editor/agent config: launch.json has absolute machine paths + settings.local.json + w**

.claude/

### 33. user (2026-07-02T02:50:08.183Z)

```
==> site/scripts/droplet-smoke.sh <==
#!/usr/bin/env bash
```

**Live VM smoke test: copy site/ to a droplet, boot server.js, round-trip it, report.**

**Proves the repo's Node + SQLite path on a REAL VM (the Cloudflare path is covered b**

**`npm test`). Idempotent; cleans up its temp DB. Needs Node >= 22 on the VM (or a**

**self-contained binary at \$REMOTE\_DIR/node/bin/node).**

```
#
SSH_TARGET=root@1.2.3.4 [SSH_KEY=~/.ssh/khelsutra_droplet] \
[REMOTE_DIR=/srv/khelsutra-site] [PORT=8787] scripts/droplet-smoke.sh
```

```
set -euo pipefail
: "${SSH_TARGET:?set SSH_TARGET=user@host}"
KEY="${SSH_KEY:-$HOME/.ssh/khelsutra_droplet}"
DIR="${REMOTE_DIR:-/srv/khelsutra-site}"
PORT="${PORT:-8787}"
SITE="$(cd "$(dirname "$0")/.." && pwd)" # the site/ folder (script lives in site/scripts/)
SSH0=(-i "$KEY" -o StrictHostKeyChecking=accept-new)

echo "? scp $SITE ? $SSH_TARGET:$DIR/site"
ssh "${SSH0[@]}" "$SSH_TARGET" "mkdir -p '$DIR'"
scp "${SSH0[@]}" -qr "$SITE" "$SSH_TARGET:$DIR/"

ssh "${SSH0[@]}" "$SSH_TARGET" "bash -s -- '$DIR' '$PORT'" <<'REMOTE'
set -e
DIR="$1"; PORT="$2"
NODE="$( [ -x "$DIR/node/bin/node" ] && echo "$DIR/node/bin/node" || command -v node || true )"
[ -n "$NODE" ] || { echo "FAIL: no Node (>=22) on the VM"; exit 1; }
cd "$DIR/site"
DB="$DIR/smoke.db"; rm -f "$DB"
PORT="$PORT" DB_PATH="$DB" nohup "$NODE" server.js > "$DIR/smoke.log" 2>&1 &
SRV=$!
for _ in $(seq 1 30); do curl -sf -o /dev/null "http://127.0.0.1:$PORT/" && break || sleep 1;
done
```

```
==> site/scripts/provision-vm.sh <==
#!/usr/bin/env bash
```

**First-class, PROVIDER-AGNOSTIC provisioning for the KhelSutra site (Node + SQLite)**  
**Run from your LOCAL machine against a FRESH Ubuntu VM (DigitalOcean / GCE / any)**  
**it pushes site/ via `git archive` (PRIVATE-REPO-SAFE ? no clone, token, or secret on t**  
**VM), installs Node, installs + starts a systemd service, opens the firewall, and VERIFIE**  
**it is actually serving. Idempotent (safe to re-run). Pairs with teardown-vm.sh so a**  
**destroy ? resume-on-a-different-provider cycle is clean and reproducible.**

```
#
SSH_TARGET=root@1.2.3.4 [SSH_KEY=~/.ssh/khelsutra_droplet] \
[REMOTE_DIR=/srv/khelsutra-site] [PORT=8787] [NODE_MAJOR=24] [RESTORE_DB:
site/scripts/provision-vm.sh
```

```
#
NODE_MAJOR must be >= 24 so `node:sqlite` is unflagged (server.js imports it directl
```

```
set -euo pipefail
: "${SSH_TARGET:?set SSH_TARGET=user@host}"
KEY="${SSH_KEY:-$HOME/.ssh/khelsutra_droplet}"
DIR="${REMOTE_DIR:-/srv/khelsutra-site}"
PORT="${PORT:-8787}"
NODE_MAJOR="${NODE_MAJOR:-24}"
SITE="$(cd "$(dirname "$0")/.." && pwd)" # the site/ dir (script lives in
site/scripts/)
REPO="$(cd "$SITE/.." && pwd)" # repo root (for git archive)
SSH0=(-i "$KEY" -o StrictHostKeyChecking=accept-new -o ConnectTimeout=10)

echo "? [1/5] waiting for SSH at $SSH_TARGET ..."
for _ in $(seq 1 60); do ssh "${SSH0[@]}" "$SSH_TARGET" true 2>/dev/null && break || sleep 5;
done
```

```
ssh "${SSH0[@]}" "$SSH_TARGET" true # final attempt ? fail loud if still down
```

```
echo "? [2/5] pushing site/ via git archive (private-repo-safe; no secret on the VM) ..."  
ssh "${SSH0[@]}" "$SSH_TARGET" "rm -rf '$DIR/site' && mkdir -p '$DIR/site'"
```

**core.autocrlf=false forces LF in the archive even when the operator runs on Windows**

#### 34. user (2026-07-02T02:50:19.269Z)

```
6f17dc0 chore(ci): bump checkout@v4?v7 and setup-node@v4?v6 (off deprecated Node 20) (#118)  
5ffb750 chore(ci): retire the live-parity monitor ? droplet mirror decommissioned (#117)  
commit 6f17dc024ae44ba309dba7290869cc1f11bd8b63  
Author: Avi <avi.dullu@gmail.com>  
Date: Thu Jun 25 17:33:35 2026 +0530
```

```
chore(ci): bump checkout@v4?v7 and setup-node@v4?v6 (off deprecated Node 20) (#118)
```

```
GitHub is force-running Node-20 actions on Node 24 and warning that Node 20 is  
deprecated (2025-09-19 changelog). `test.yml` pinned `actions/checkout@v4` and  
`actions/setup-node@v4`, both Node-20 runtimes. Bump to the current majors  
(checkout v7.0.0, setup-node v6.4.0 ? both Node 24). The `node-version: "24"`  
input and `cache` settings are unchanged (setup-node@v6 keeps those inputs).
```

```
(The other source of this warning ? the parity workflow ? was removed in #117.)
```

```
Co-authored-by: Claude Opus 4.8 <noreply@anthropic.com>
```

```
.github/workflows/test.yml | 12 ++++++-----  
1 file changed, 6 insertions(+), 6 deletions(-)
```

#### 35. user (2026-07-02T02:50:20.939Z)

No matches found

#### 36. user (2026-07-02T02:50:32.347Z)

```
#!/usr/bin/env bash
```

**Bundle the KhelSutra web SPA into the engine so a downloaded engine serves the ready-to-run SPA (single-origin (PACKAGING\_PLAN.md P1b)).**

```
#
```

**OPTION 1 (now, owner-chosen): build the SPA from a khelsutra checkout and vendor it into `backend/ui`, which is COMMITTED ? so the wheel + CI build fully offline/reproducible. The snapshot by re-running this on an SPA release; commit the diff.**

```
#
```

**OPTION 3 (designed-for, future): point --from at a published dist tarball/URL (a GitHub artifact) to vendor a pre-built SPA without a local khelsutra checkout or Node. Already done.**

```
#
```

**Usage:**

**scripts/bundle\_ui.sh [SOURCE]**

**SOURCE = a khelsutra checkout dir (default: \$KHELSUTRA\_DIR or ../khelsutra) [builds SPA] | a path/URL ending .tgz/.tar.gz [unpacks it]**

**BUILD=0 scripts/bundle\_ui.sh <khelsutra-dir> # skip npm; reuse an existing web/dist**

```
set -euo pipefail
```

```
ENGINE_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"  
UI_DEST="$ENGINE_ROOT/backend/ui"
```

```

SOURCE="${1:-${KHELSUTRA_DIR:-$ENGINE_ROOT/../khelesutra}}"

stage_dist() { # populates $1 (a dist dir) into a clean $UI_DEST
  local dist="$1"
  [ -f "$dist/index.html" ] || { echo "ERROR: no index.html in $dist ? not a built SPA dist"
>&2; exit 1; }
  rm -rf "$UI_DEST"
  mkdir -p "$UI_DEST"
  cp -r "$dist"/. "$UI_DEST"/
}

SPA_REPO="unknown"
SPA_COMMIT="unknown"

case "$SOURCE" in
  *.tgz|*.tar.gz|http://*|https://*)
    # OPTION 3: a published dist artifact.
    tmp="$(mktemp -d)"; trap 'rm -rf "$tmp"' EXIT
    if [[ "$SOURCE" == http*://* ]]; then
      echo "[bundle_ui] fetching dist artifact: $SOURCE"
      curl -fsSL "$SOURCE" -o "$tmp/dist.tgz"; SRC_TGZ="$tmp/dist.tgz"; SPA_REPO="$SOURCE"

```

### 37. user (2026-07-02T02:50:33.743Z)

```

269:   Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
guard).""
408:API_VERSION = 1
424:   "api_version": API_VERSION,
450:   "api_version": API_VERSION,

```

### 38. user (2026-07-02T02:50:51.197Z)

```

README.md
annotate.pw.mjs
fixtures
ingest.pw.mjs
mock-engine.mjs
playwright.config.mjs
screencast.pw.mjs
store-medium.pw.mjs
to-mp4.mjs
====
====ISSUES====
114   OPEN    Cloudflare Access: an allowlisted email still couldn't sign in (alpha auth)
2026-06-23T10:25:52Z
112   OPEN    Share to WhatsApp sends only a text message, not the reel video bug
2026-06-22T18:27:52Z
111   OPEN    Resilient processing UX: survive page refresh + continually-updating progress
enhancement    2026-06-22T18:15:04Z
103   OPEN    SPA: show video stats (name, size, fps, resolution, duration) + persist them on
reload         2026-06-22T14:18:41Z
102   OPEN    SPA: &debug=1 (trace panel) / &debug=2 (full runlog) URL params for debugging +
tracing        2026-06-22T14:16:43Z
101   OPEN    SPA: show a friendly failure notice on a timeout/5xx (Cloudflare 524), not the
raw Cloudflare error HTML    2026-06-22T14:14:16Z
99    OPEN    SPA: keep the uploaded video's filename visible during processing (it disappears
mid-job)        2026-06-22T14:01:52Z
98    OPEN    SPA: replace the demo rally-selection the moment an upload/processing starts (so
the user knows their reel is coming)    2026-06-22T13:57:56Z
97    OPEN    Hosted alpha: split-origin (Pages app. + tunnel api.) breaks Cloudflare Access
for the SPA ? single-origin needed    2026-06-22T13:54:39Z
72    OPEN    i18n: add Telugu (te) locale (catalog + Noto Telugu font, key-parity green)
2026-06-21T18:17:34Z

```

### 39. assistant (2026-07-02T02:51:15.634Z)

No open PRs; no ESLint config in web/. Now checking the just-landed #140 code for latent issues.

### 40. user (2026-07-02T02:51:16.677Z)

```
1      // Resilient readers for a GET /api/videos row (VideoSummary). Defense-in-depth against
engine drift:
2      // the engine SHOULD return the contract fields (`video_id`, a friendly `match_name`,
`fps`,
3      // `resolution`), but if a build regresses to the raw DB shape (`id`, `filepath`, no
name), the library
4      // + the persistent current-video label still work ? they read through these helpers,
never the raw
5      // field. (The bug this guards against actually shipped once: the real engine returned
`id`/no-name
6      // while the e2e mock faked both, so a "green" UI was broken live.)
7      import type { VideoSummary } from "../api/types";
8
9      /** The MD5 join key ? falls back to the raw `id` column if `video_id` is absent. */
10     export function videoId(v: VideoSummary): string {
11         return v.video_id ?? v.id ?? "";
12     }
13
14     /** Filename stem of a path/URI: `~/My Match 2026.mp4` ? `My Match 2026`. Handles `~`
and `\\`,
15     * a trailing slash, and a no-extension name; returns "" for empty input. */
16     export function basenameStem(path: string | null | undefined): string {
17         const base =
18             String(path ?? "")
19                 .replace(/[/\\]+$/, "")
20                 .split(/[/\\]/)
21                 .pop() ?? "";
22         const dot = base.lastIndexOf(".");
23         return dot > 0 ? base.slice(0, dot) : base;
24     }
25
26     /** A human name ? the engine's `match_name`, else derived from the stored path/URI,
else "" (the
27     * caller applies its own localized "unnamed"/"your video" fallback). */
28     export function videoName(v: VideoSummary): string {
29         return v.match_name || basenameStem(v.local_path ?? v.filepath);
30     }
31
32     /** A compact spec line, e.g. `1920x1080 · 30fps` (either part optional; "" when neither
is known). */
33     export function videoSpec(v: VideoSummary): string {
34         const parts: string[] = [];
35         if (v.resolution) parts.push(v.resolution);
36         if (typeof v.fps === "number" && v.fps > 0) parts.push(`${Math.round(v.fps)}fps`);
37         return parts.join(" · ");
38     }
39
```

### 41. user (2026-07-02T02:51:17.140Z)

```
1      // "Your videos" ? the indexed-video library (STUDIO_MEDIA_LIFECYCLE.md P6). Consumes
the engine's
2      // GET /api/videos so a match already added can be REOPENED without re-processing
(reload-survival),
3      // and an ingest?annotate loop across many videos has a home. Honest + quiet: on loading
/ error /
4      // empty it stays out of the way (the ingest panel below is the primary "add a new one"
path).
5      import { useEffect, useState } from "react";
```

```

6     import { useTranslation } from "react-i18next";
7     import { listVideos } from "../api/client";
8     import type { VideoSummary } from "../api/types";
9     import { log, newCorrelationId } from "../lib/log";
10    import { videoId, videoName, videoSpec } from "../lib/video";
11
12    type State =
13      | { status: "loading" }
14      | { status: "loaded"; videos: VideoSummary[] }
15      | { status: "error" };
16
17    interface Props {
18      /** Called when the user reopens a stored video ? with its id and human name (for the
19       * persistent current-video label, so App needn't re-resolve it). */
20      onSelect: (videoId: string, name: string) => void;
21    }
22
23    export function VideoLibrary({ onSelect }: Props) {
24      const { t } = useTranslation();
25      const [state, setState] = useState<State>({ status: "loading" });
26
27      useEffect(() => {
28        let alive = true;
29        const cid = newCorrelationId();
30        listVideos(cid)
31          .then((videos) => alive && setState({ status: "loaded", videos }))
32          .catch((err) => {
33            if (!alive) return;
34            // Quiet, non-blocking: the library is a convenience ? a load failure must not
hide the
35            // ingest panel. Log it (observability) but degrade to a soft message.
36            setState({ status: "error" });
37            log("warn", "library.load_failed", { cid, error: String(err) });
38          });
39        return () => {
40          alive = false;
41        };
42      }, []);
43
44      const Title = <h2 className="text-lg font-bold">{t("library.title")}</h2>;
45
46      if (state.status === "loading") {
47        return (
48          <section className="mt-6" data-testid="video-library" aria-busy="true">
49            {Title}
50            <p className="mt-2 text-sm text-ink/50">{t("library.loading")}</p>
51          </section>
52        );
53      }
54      if (state.status === "error") {
55        return (
56          <section className="mt-6" data-testid="video-library">
57            {Title}
58            <p className="mt-2 text-sm text-ink/50">{t("library.errorLoad")}</p>
59          </section>
60        );
61      }
62      if (state.videos.length === 0) {
63        return (
64          <section className="mt-6" data-testid="video-library">
65            {Title}
66            <p className="mt-2 text-sm text-ink/50">{t("library.empty")}</p>
67          </section>
68        );
69      }

```



```

70
71     return (
72         <section className="mt-6" data-testid="video-library">
73             {Title}
74             <p className="mt-1 text-sm text-ink/60">{t("library.intro")}</p>
75             <ul className="mt-3 grid grid-cols-[repeat(auto-fill,minmax(220px,1fr))] gap-3
list-none p-0 m-0">
76                 {state.videos.map((v, i) => {
77                     const id = videoId(v);
78                     const name = videoName(v) || t("library.unsigned");
79                     const meta = [videoSpec(v), v.created_at].filter(Boolean).join(" · ");
80                     return (
81                         <li key={id || `row-${i}`}>
82                             <button
83                                 type="button"
84                                 onClick={() => onSelect(id, name)}
85                                 aria-label={t("library.openAria", { name })}
86                                 data-testid="library-item"
87                                 className="w-full text-left rounded-xl border border-black/10 bg-white
p-3 transition hover:border-green focus:outline-none focus:ring-2 focus:ring-green"
88                             >
89                                 <div className="truncate font-semibold text-ink">{name}</div>
90                                 {meta && <div className="mt-1 truncate text-xs text-
ink/50">{meta}</div>}
91                                 <span className="mt-2 inline-flex items-center text-sm font-semibold
text-green">
92                                     {t("library.open")} ?
93                                 </span>
94                             </button>
95                         </li>
96                     );
97                 })}
98             </ul>
99         </section>
100     );
101 }
102

```

## 42. user (2026-07-02T02:51:35.326Z)

```

// Mock engine ? a dependency-free Node HTTP server implementing the badminton-highlight-indexer
// contract the Studio SPA calls, PLUS serving the built SPA from ./dist. One origin, so the
SPA's
// default same-origin `/api/*` calls resolve here exactly as they would behind the appliance
reverse
// proxy. This is the hermetic L1 layer from docs/LOCAL_APPLIANCE_ARCHITECTURE.md §8 ?
deterministic,
// no GPU/key/Python ? used to record the e2e screencast and (later) drive CI CUJ replays.
//
// It is a MOCK: fixtures are canned and the job state machine advances by poll count (one
"running"
// poll, then "done") so a screencast shows real progress. The response SHAPES mirror the
verified
// engine contract (web/src/api/types.ts + client.ts) so the SPA drives it unmodified.
import http from "node:http";
import { readFile } from "node:fs/promises";
import { existsSync } from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const WEB_ROOT = path.resolve(__dirname, ".."); // khelsutra/web
const DIST = path.join(WEB_ROOT, "dist");
const REEL = path.join(__dirname, "fixtures", "reel.mp4");
const PORT = Number(process.env.MOCK_PORT || 8791);

```

```

const MIME = {
  ".html": "text/html; charset=utf-8", ".js": "text/javascript; charset=utf-8",
  ".mjs": "text/javascript; charset=utf-8", ".css": "text/css; charset=utf-8",
  ".json": "application/json; charset=utf-8", ".svg": "image/svg+xml", ".png": "image/png",
  ".jpg": "image/jpeg", ".jpeg": "image/jpeg", ".ico": "image/x-icon", ".webp": "image/webp",
  ".woff": "font/woff", ".woff2": "font/woff2", ".ttf": "font/ttf", ".mp4": "video/mp4",
  ".webm": "video/webm", ".map": "application/json",
};

// Deterministic rally fixtures ? the verified play_segments shape (honest fields only). id 4
sits
// below the mock's min_shot_count (5) on purpose, so the SPA's honest floor-warning is
captured.
const RALLIES = [
  { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null, metadata:
{ shot_count: 9, shot_count_confidence: "High" } },
  { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null, metadata:
{ shot_count: 6 } },
  { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null, metadata:
{ shot_count: 19, shot_count_confidence: "Medium" } },
  { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null, metadata:
{ shot_count: 4 } },
  { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null, metadata:
{ shot_count: 11 } },
  { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null, metadata:
{ shot_count: 24, shot_count_confidence: "High" } },
];

const CAPS = {
  api_version: 1,
  engine_version: "mock-0.1.0",
  segmenters: ["gemini", "motion", "wasb_hybrid"],
  default_segmenter: "gemini",
  ffmpeg: { ffmpeg: true, ffprobe: true, ffmpeg_path: "/usr/bin/ffmpeg", ffprobe_path:
"/usr/bin/ffprobe" },
  gpu: { nvidia_smi_present: false },
  gemini_key_present: true,
  wasb_weights_present: false,
  deployment: { profile: "local-appliance", compute_target: "local", cloud_available: false },
  // Forward-compatible storage block (STUDIO_MEDIA_LIFECYCLE.md P3/P5). The current SPA ignores
  // unknown fields; the medium picker will render from `storage.backends` once P3/P5 land.
  storage: {
    default_medium: "local",
    backends: [
      { id: "local", kind: "local", label: "This computer", usable: true,
free_gb: 812 },
      { id: "gcs", kind: "gcs", label: "Google Cloud Storage bucket", usable: true,
free_gb: null },
      { id: "s3", kind: "s3", label: "Amazon S3 / compatible bucket", usable: true,
free_gb: null },
      { id: "gdrive", kind: "gdrive", label: "Google Drive folder (mounted)", usable: true,
free_gb: 1900 },
    ],
  },
  ===

```

## Vendored: rally-annotator annotation core (MIT)

`csv.ts` and `annotator.ts` here are **vendored verbatim** from the open-source, MIT-licensed **[[rally-annotator](https://github.com/KheIsutra/rally-annotator)]** (``web/src/state/``), synced at commit `3d604bc`. They are the pure, framework-free labeling core ? the rally **CSV** (``RallyRow`` + `serialize/pars`e, byte-compatible with the VLC plugin and the browser extension) and the `Annotator`` state machine (mark start/end · reason · shots · save/edit/delete/undo, with an injectable `PersistFn``). No ``chrome`/`wxt`/DOM` coupling.

## Why vendored (not an npm dependency)

Studio embeds the labeler as an in-browser React panel (`STUDIO\_MEDIA\_LIFECYCLE.md` D10/D13).  
Vendoring  
the *\*pure core\** ? rather than publishing a library from the public repo and depending on it ?  
keeps the  
public MIT repo *\*\*untouched\*\** (no Studio glue leaks in), avoids cross-repo release coupling, and  
lets  
Studio build a native React UI + inject its own video handler and persist (POST to the engine).  
MIT  
permits this use in a proprietary product provided the copyright + license notice travel with  
the copy  
(below).

## Rules

- *\*\*Do not edit these two files here.\*\** Fix upstream in rally-annotator, then re-sync (update the commit above). Keeping them byte-compatible is what lets the labels ingest into the same pipeline (`sports-data-collector` ? `badminton-highlight-indexer`), joined by the video's file MD5.
- The Studio-specific glue (video handler, POST-to-engine persist, save-state UI, Studio strings) lives in `web/src/components/AnnotatePanel.tsx`, *\*\*not\*\** here.

## License (MIT ? from rally-annotator/LICENSE)

...

MIT License

Copyright (c) 2026 Avi Dullu

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

...

### 43. user (2026-07-02T02:51:45.392Z)

5ec1831 feat(web): i18n infrastructure + English catalog (keys-not-prose) (#18)  
ad6ac9c Add rally-annotator web extension (Chrome MV3) with proof-driven tests + CI (#14)  
3d604bc docs: sync READMEs with current state (multi-agent audit fixes) (#25)  
3d604bc is ancestor

### 44. user (2026-07-02T02:51:46.735Z)

site/wrangler.toml:1:# Cloudflare Worker for khelsutra.guru ? static site + Request-Access form API.  
site/wrangler.toml:8:name = "hidden-king-khelsutra" # the git-connected Worker that owns khelsutra.guru ? keep in sync with Cloudflare  
site/wrangler.toml:27:NOTIFY\_FROM = "noreply@khelsutra.guru"  
docs/DRY\_RUN\_CUJ.md:3:> *\*\*Status:\*\** `DONE ? DELIVERED (2026-06-21)` . *\*\*Owner:\*\** Avi Dullu .  
*\*\*Created:\*\** 2026-06-21. DoD §6 boxes 1?3 ? (Path A + Path B/B-P2 live e2e); the only open box

(droplet, \$6 box 4) is **\*\*owner-parked\*\*** (GPU subscription). Archive once that box closes.  
docs/DRY\_RUN\_CUJ.md:30:1. **\*\*Engine host + deps:\*\*** run the engine FastAPI (`python -m backend.main --server`, binds `127.0.0.1:8000`) on a box with `ffmpeg` + OpenCV. Gemini/motion paths are **\*\*CPU-only, no WSL\*\*** ? local Windows works; the CPU droplet works too. (GPU/WSL only needed for native WASB.)  
docs/DRY\_RUN\_CUJ.md:42:3. **\*\*Engine host:\*\*** local Windows (simplest, Gemini/motion) · the CPU droplet `168.144.126.176` (shared testbed ? careful) · a GPU VM (only for WASB).  
docs/DRY\_RUN\_CUJ.md:56:- [ ] **\*(option 3)\*** the same CUJ confirmed on the **\*\*CPU droplet\*\*** (Linux deploy target) ? droplet reachable but currently **\*\*un-provisioned\*\*** (no engine checkout; ~3.8GB RAM) ? needs a from-scratch engine install or a proper deploy of the fixed engine.  
docs/DRY\_RUN\_CUJ.md:69:The async-job server **\*\*hung every job in `queued`\*\*** when run the documented way (`python -m backend.main`): main.py runs as `\_\_main\_\_` (sets `\_\_main\_\_.db\_instance`), but `job\_worker` reads `import backend.main` ? a **\*\*separate\*\*** module whose `db\_instance` is `None` ? the drain crashes silently. Latent because the unit tests monkeypatch the executor inline and the UI was verified vs a stub engine. **\*\*Fix + a real-subprocess regression test ? engine [281](https://github.com/Khelsutra/badminton-highlight-indexer/pull/281)\*\*** (dispatch through `backend.main`; FAILS-without / PASSES-with). Unblocked the run via the single-module invocation; the fix applies to the droplet too.

#### 45. assistant (2026-07-02T02:52:04.494Z)

Let me disambiguate the rally-annotator vendor-sync state.

#### 46. user (2026-07-02T02:52:07.250Z)

HEAD:  
3d604bc docs: sync READMEs with current state (multi-agent audit fixes) (#25)  
state-touching commits after 3d604bc:  
diff stat:  
(empty = in sync)

#### 47. user (2026-07-02T02:52:21.909Z)

```

727     @app.post("/api/assets", status_code=201)
728     def create_asset(req: AssetRequest, request: Request):
729         """Store a video into the user's chosen medium and register it as a library asset
(P4, D14).
730
731         SELF-HOST posture only (D7): refused with 403 when ``security.allowed_video_root``
is set ? the
732         hosted alpha jails inputs and has no per-user medium credentials, so it stays
upload-to-local.
733         Edge-auth mirrors ``/api/process``. Exactly one source: an ``uploaded_path`` (a
local file from
734         the chunked upload) OR a ``source_uri`` (a video already at rest in a medium). An
uploaded file +
735         a REMOTE medium ? a durable, MD5-keyed copy is ``put`` into the medium while the
local upload
736         stays as the working copy; a ``local`` medium keeps the upload in place (it's
already on this box);
737         a pasted at-rest URI is registered as-is (no copy, D14b). The asset is keyed by the
whole-file
738         MD5 (``compute_video_id``, D3) so it joins the pipeline by content hash, and appears
in
739         ``GET /api/videos``.
740         try:
741             owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
742         except edge_auth.EdgeAuthError as e:
743             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
744
745         cfg = request.app.state.config
746         sec = getattr(cfg, "security", None)
747         if getattr(sec, "allowed_video_root", None):
748             raise HTTPException(

```

```

749         status_code=403,
750         detail=(
751             "storing into a custom medium is disabled on this hosted deployment "
752             "(security.allowed_video_root is set); upload to local instead."
753         ),
754     )
755 if bool(req.uploaded_path) == bool(req.source_uri):
756     raise HTTPException(
757         status_code=400,
758         detail="provide exactly one of 'uploaded_path' or 'source_uri'.",
759     )
760 if req.uploaded_path and not req.medium_uri:
761     raise HTTPException(
762         status_code=400, detail="'medium_uri' is required when uploading a file."
763     )
764
765 storage_settings = _storage_settings(cfg)
766
767 # 1) Resolve a LOCAL working copy of the source + its whole-file MD5 (the join key,
D3).
768 if req.uploaded_path:
769     try:
770         validate_input_path(req.uploaded_path, allowed_root=None)
771         src_ref = storage.resolve_input_ref(req.uploaded_path, storage_settings)
772     except ValueError as e:
773         raise HTTPException(status_code=400, detail=str(e)) from e
774     if src_ref.backend != "local" or not os.path.isfile(src_ref.key):
775         raise HTTPException(
776             status_code=404,
777             detail=f"uploaded file not found at '{req.uploaded_path}'",
778         )
779     local_working = src_ref.key
780 else:
781     try:
782         validate_input_path(req.source_uri, allowed_root=None)
783         src_ref = storage.resolve_input_ref(req.source_uri, storage_settings)
784     except ValueError as e:
785         raise HTTPException(status_code=400, detail=str(e)) from e
786     # A remote at-rest source is fetched to scratch to hash it (D3) ? reserve that
scratch (P2).
787     if src_ref.backend != "local":
788         need = _remote_input_size_bytes(src_ref, getattr(cfg, "storage", None))
789         require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
790     try:
791         local_working = storage.acquire_local_input(
792             req.source_uri, getattr(cfg, "storage", None)
793         )
794     except ValueError as e:
795         raise HTTPException(status_code=400, detail=str(e)) from e
796     except Exception as e: # noqa: BLE001 ? a fetch miss is a 502, surfaced not
swallowed
797         raise HTTPException(
798             status_code=502, detail=f"could not resolve source_uri: {e}"
799         ) from e
800     if not os.path.isfile(local_working):
801         raise HTTPException(
802             status_code=404, detail="source_uri did not resolve to a readable file."
803         )
804
805 video_id = compute_video_id(local_working)
806 ext = os.path.splitext(local_working)[1] or ".mp4"
807
808 # 2) Decide the durable location + copy only when needed.
809 copied = False
810 if req.source_uri:

```

811                    stored\_uri = req.source\_uri # already at rest in its medium ? no copy (D14b)

#### 48. user (2026-07-02T02:52:23.644Z)

```
{"state":"OPEN","title":"refactor(D1/P10): extract orchestration.py from main.py (god-module split, step 1)","updatedAt":"2026-06-26T14:37:18Z"}
```

#### 49. user (2026-07-02T02:52:36.395Z)

No matches found

#### 50. user (2026-07-02T02:52:37.923Z)

```
web/src/api/client.ts:87: * processing. POST /api/process {video_path} ? 202 {job_id, ...}
[main.py:260]. Fast-fails 400
web/src/api/client.ts:105:/** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ?
unknown job_id. */
web/src/api/client.ts:110:/** List indexed videos, newest first ? the "my videos" library. GET
/api/videos [main.py:447].
web/src/api/client.ts:156:/** List a video's detected rallies. POST /api/query {video_id} ?
{play_segments} [main.py:331]. */
web/src/api/client.ts:223:/** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
web/src/api/client.ts:228:/** Read engine config (for the stitching floor). GET /api/config
[main.py:121]. */
web/src/api/client.ts:238:/** The shot-count floor /api/compile silently enforces ? the UI must
warn, not hide [main.py:362]. */
web/src/api/client.ts:247:/** Start a chunked upload. 400 = bad extension; 413 = over the size
limit [uploads.py:64]. */
web/src/api/client.ts:256:/** Append ONE part (raw bytes ? not JSON). Parts are strictly
sequential 0,1,2,? [uploads.py:89]. */
web/src/api/client.ts:288:/** Finalize: returns {path, video_id} ? feed `path` to processVideo()
[uploads.py:123]. */
web/src/api/client.ts:297:/** Abort an in-flight upload (best-effort cleanup of the server-side
partial) [uploads.py:147]. */
web/src/api/types.ts:3:// backend/main.py:331-396 (query/compile), :307-328 (highlights), :121
(config).
web/src/api/types.ts:33:/** One row of the indexed-video library. GET /api/videos
[backend/main.py:447] returns the most
web/src/api/types.ts:143:/** POST /api/process ? 202 [backend/main.py:260]. The job is queued;
poll GET /api/jobs/{job_id}. */
web/src/api/types.ts:172:/** GET /api/jobs/{job_id} [backend/main.py:303]. Two status layers:
```

#### 51. user (2026-07-02T02:53:12.359Z)

```
tracked-count: 0
web/dist
ignored
```

#### 52. user (2026-07-02T02:53:13.178Z)

```
web/src/api/client.test.ts:230: vi.stubGlobal("fetch", vi.fn(async () => { throw new
Error("offline"); }) as unknown as FetchFn);
web/src/components/a8.status.test.tsx:102: throw new Error("offline");
web/src/components/build-error.test.tsx:58: throw new Error("offline");
web/src/components/build-error.test.tsx:92: if (u === "/api/query" && method === "POST")
throw new Error("offline"); // engine down on load
web/src/components/FfmpegBanner.test.tsx:64: throw new Error("offline");
web/src/components/FfmpegBanner.test.tsx:9:// engine doesn't report the ffmpeg block at all, it stays
hidden ? the offline banner / wizard cover
web/src/components/IngestPanel.test.tsx:114: throw new Error("offline");
web/src/components/QueryBar.test.tsx:4:// live parsed-pill, and the "unreadable" branch were
untested (funcs 40%). The offline parser
```

### 53. user (2026-07-02T02:53:25.338Z)

0  
14 annotate  
10 medium  
8bcf6ce feat(api): make /api/compile async (202 + poll) to kill the slow-stitch 524 (#329) (#339)  
293e5c0 feat(pkg): re-bundle backend/ui ? serve the P2c first-run wizard from indexer --app (#312)  
4a06b8b feat(pkg): P1b ? bundle the KhelSutra SPA single-origin into the engine wheel (default-OFF) (#302)

### 54. assistant (2026-07-02T02:54:14.194Z)

Let me quickly check CI status on khelsutra master and the issue-overlap evidence for #99/#103.

### 55. user (2026-07-02T02:54:23.942Z)

completed success web: resilient video-summary readers + show fps in the library (#140)  
test master push 28532935786 1m28s 2026-07-01T16:38:03Z  
completed success docs: refresh khelsutra docs for today's storage-track work (P2-P5) (?  
test master push 28530208455 1m29s 2026-07-01T15:51:14Z  
completed success web: persistent "FFmpeg not detected" banner (hard-blocker warning) (?  
test master push 28530204391 1m14s 2026-07-01T15:51:10Z  
====  
OPEN SPA: keep the uploaded video's filename visible during processing (it disappears mid-job)  
OPEN SPA: show video stats (name, size, fps, resolution, duration) + persist them on reload

### 56. user (2026-07-02T02:55:34.880Z)

Structured output provided successfully